

3D visualization and processing with Napari

Thierry Pécot

Research Engineer

Biosit SFR UMS CNRS 3480 – Inserm 018

CZI Imaging Scientist



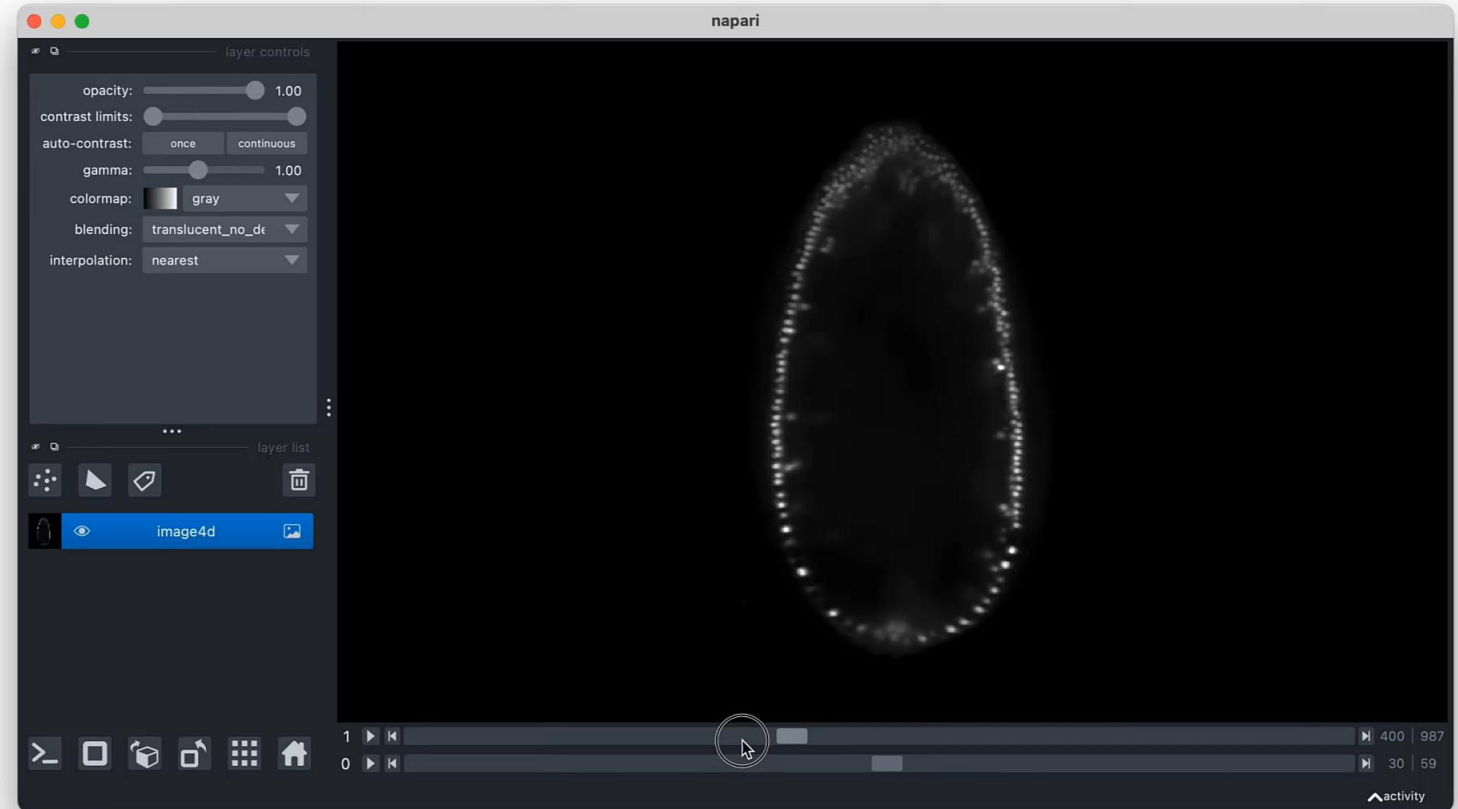
Université
de Rennes



🔍 search

Ctrl K

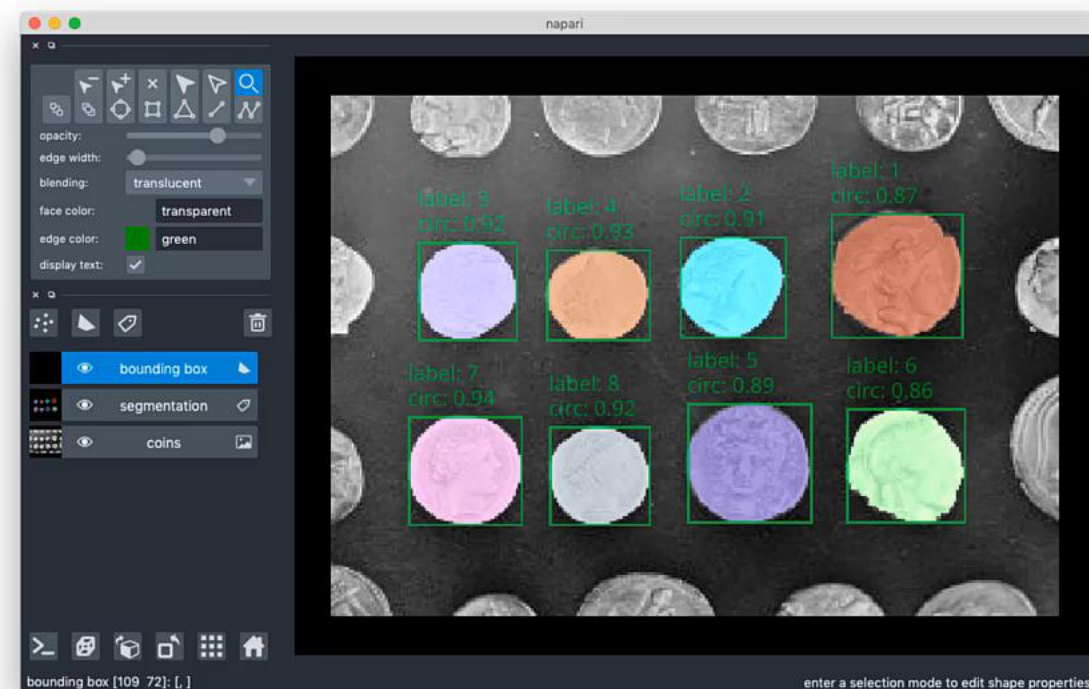
napari: a fast, interactive viewer for multi-dimensional images in Python



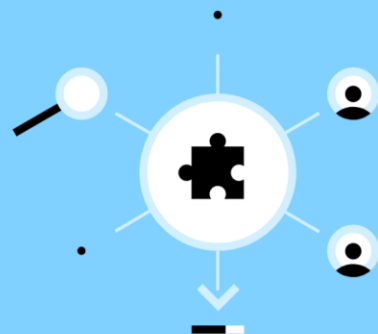
Annotating segmentation with text and bounding boxes

In this tutorial, we will use napari to view and annotate a segmentation with bounding boxes and text labels. Here we perform a segmentation by setting an intensity threshold with Otsu's method, but this same approach could also be used to visualize the results of other image processing algorithms such as object detection with neural networks.

ON THIS PAGE

[Segmentation](#)
[Analyzing the segmentation](#)
[Visualizing the segmentation results](#)
[Annotating shapes with text](#)
[Summary](#)


Discover, install, and share napari plugins



- Discover plugins that solve your image analysis challenges
- Learn how to install into napari

Share your image analysis tools with napari's growing community

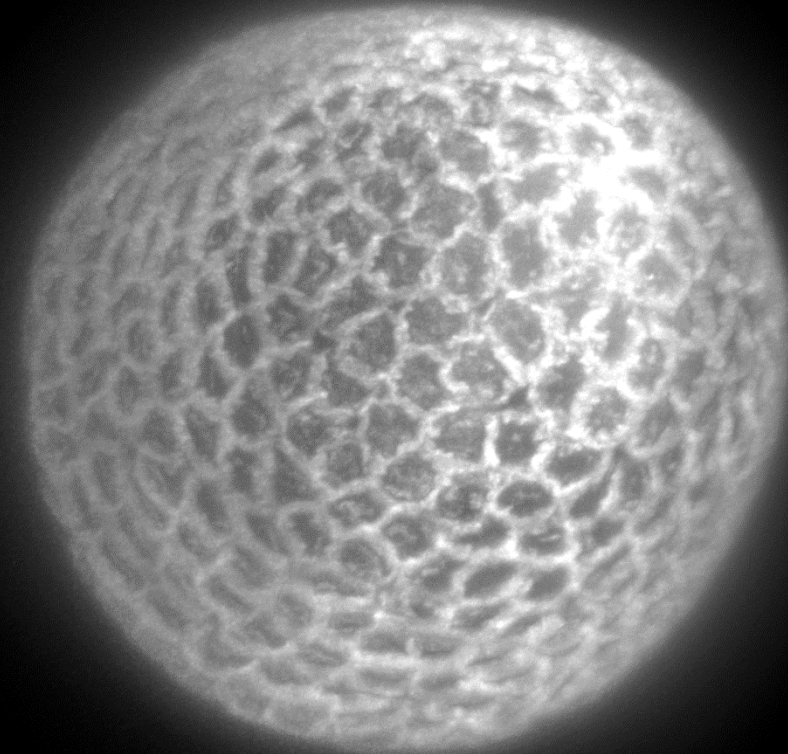
Discover Plugins [Browse all](#)

Search for a plugin by keyword or author



In an **Anaconda** prompt:

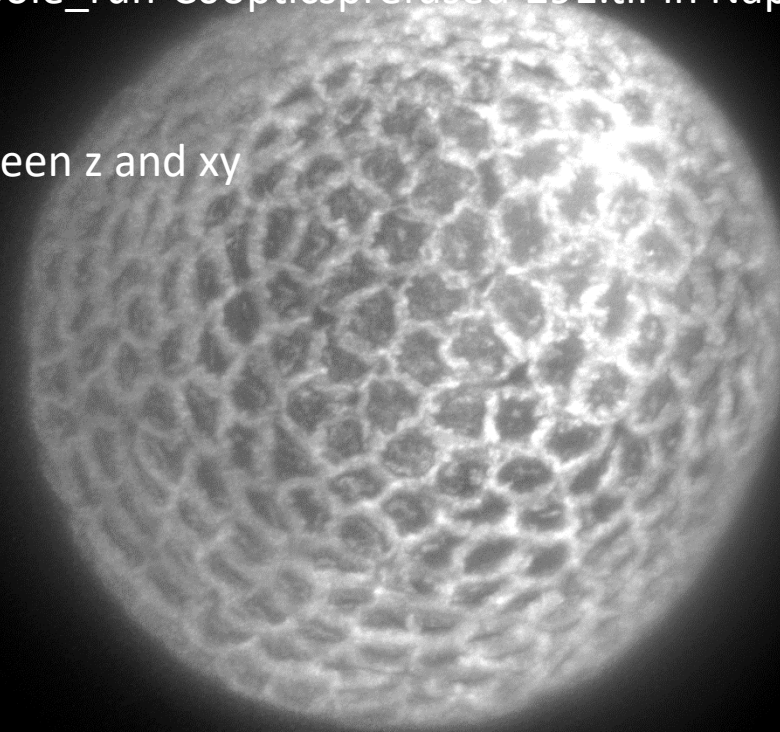
- `conda create -n NapariEnv python=3.10 openjdk=8` // create a virtual environment called NapariEnv
- `conda activate NapariEnv` // activate the virtual environment NapariEnv
- `pip install napari[all]` // install Napari
- `napari` // open Napari



In an **Anaconda** prompt:

- `conda create -n NapariEnv python=3.10 openjdk=8` // create a virtual environment called NapariEnv
- `conda activate NapariEnv` // activate the virtual environment NapariEnv
- `pip install napari[all]` // install Napari
- `napari` // open Napari

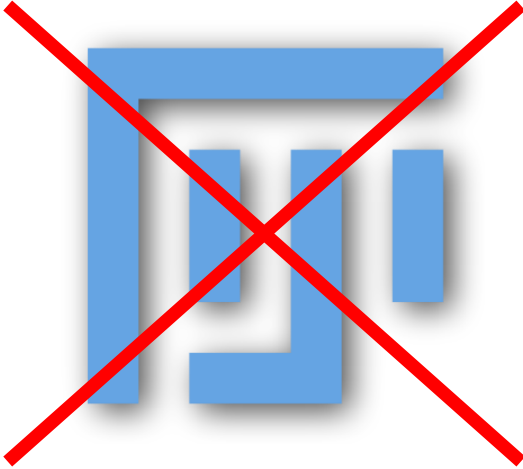
- Drag and drop Strausberg_Tribolium_LA-GFP_tailpole_run-C0opticsprefused-291.tif in Napari (available at <https://zenodo.org/records/3981193>)
- Visualize image in 2D and 3D, change scaling between z and xy
- Change contrast, change colormap, blending, ...
- Add Points layer
- Add shape layer
- Add a label layer, annotate 2 cells



Visualization



Visualization



- The **whole image** is loaded into the computer's **RAM** memory when opened with Fiji
- Stacks acquired with **lightsheet** microscopes = tens of GB

Visualization



- Open as virtual stack
- Save and visualize it with BigDataViewer

Visualization



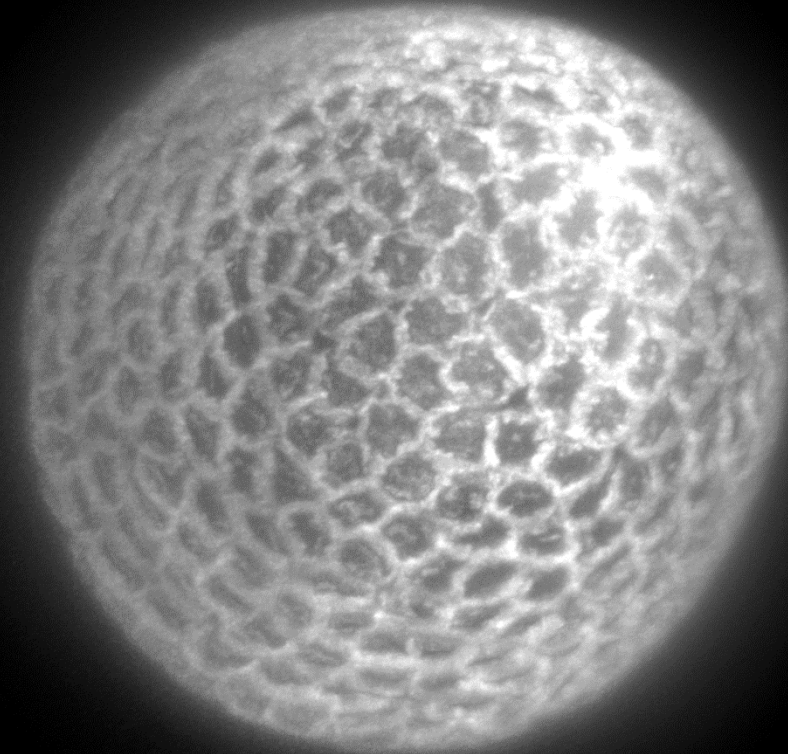
- Open as virtual stack
- Save and visualize it with BigDataViewer



- Zarr format **splits data into chunks** so only **one or several chunks** are loaded **at once** into the computer's **RAM** memory
- **Napari**, a Python visualizer, allows to **visualize zarr data**

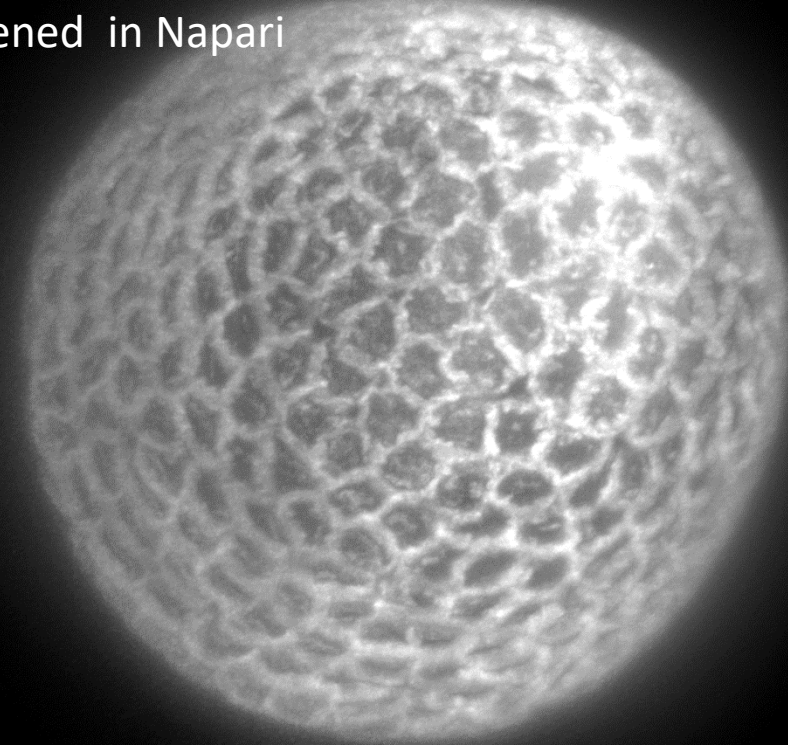
In a Miniforge **prompt**:

- `conda install -c ome bioformats2raw` // install Python package bioformats2raw
- `bioformats2raw Strausberg_Tribolium_LA-GFP_tailpole_run-C0opticsprefused-291.tif Strausberg_Tribolium_LA-GFP_tailpole_run-C0opticsprefused-291.zarr` // convert to zarr
- `pip install napari-ome-zarr` // install Napari plugin to open zarr images



In a Miniforge **prompt**:

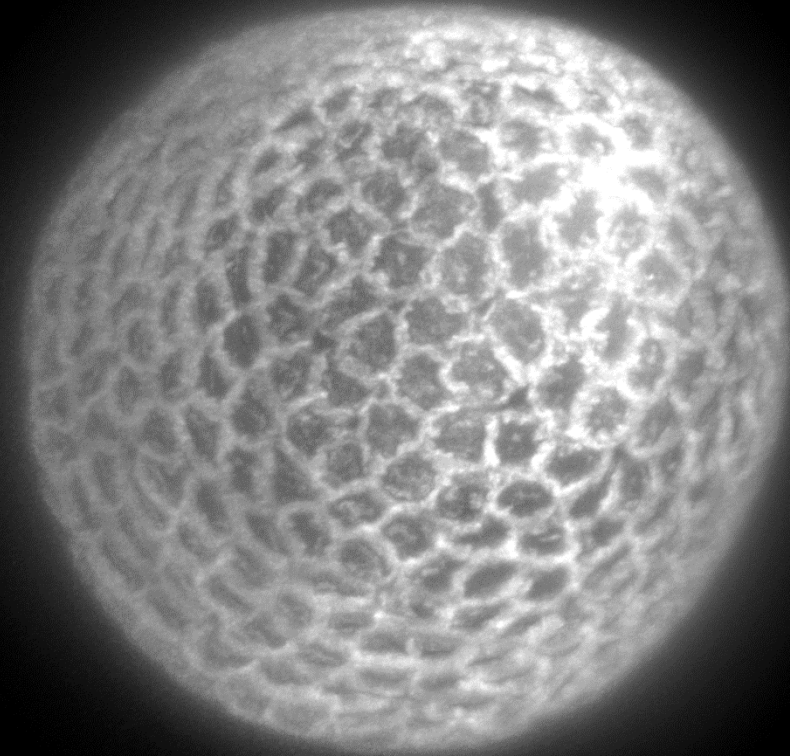
- `conda install -c ome bioformats2raw` // install Python package bioformats2raw
 - `bioformats2raw Strausberg_Tribolium_LA-GFP_tailpole_run-C0opticsprefused-291.tif Strausberg_Tribolium_LA-GFP_tailpole_run-C0opticsprefused-291.zarr` // convert to zarr
 - `pip install napari-ome-zarr` // install Napari plugin to open zarr images
-
- Compare the RAM usage between the tif and the zarr version of Strausberg_Tribolium_LA-GFP_tailpole_run-C0opticsprefused-291 when opened in Napari



In a Miniforge **prompt**:

- `pip install napari-animation`

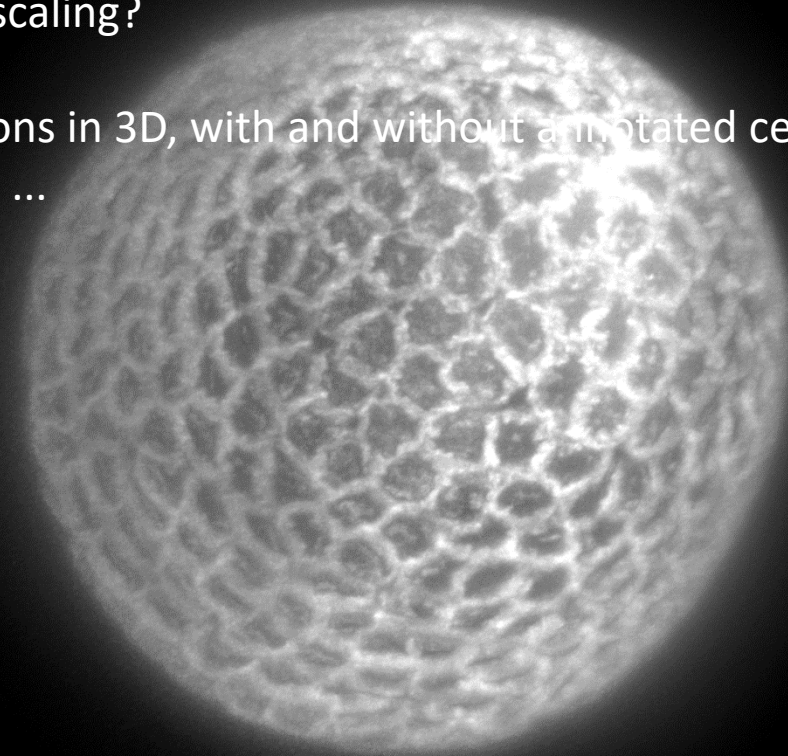
`// install Napari plugin to create videos`



In a Miniforge **prompt**:

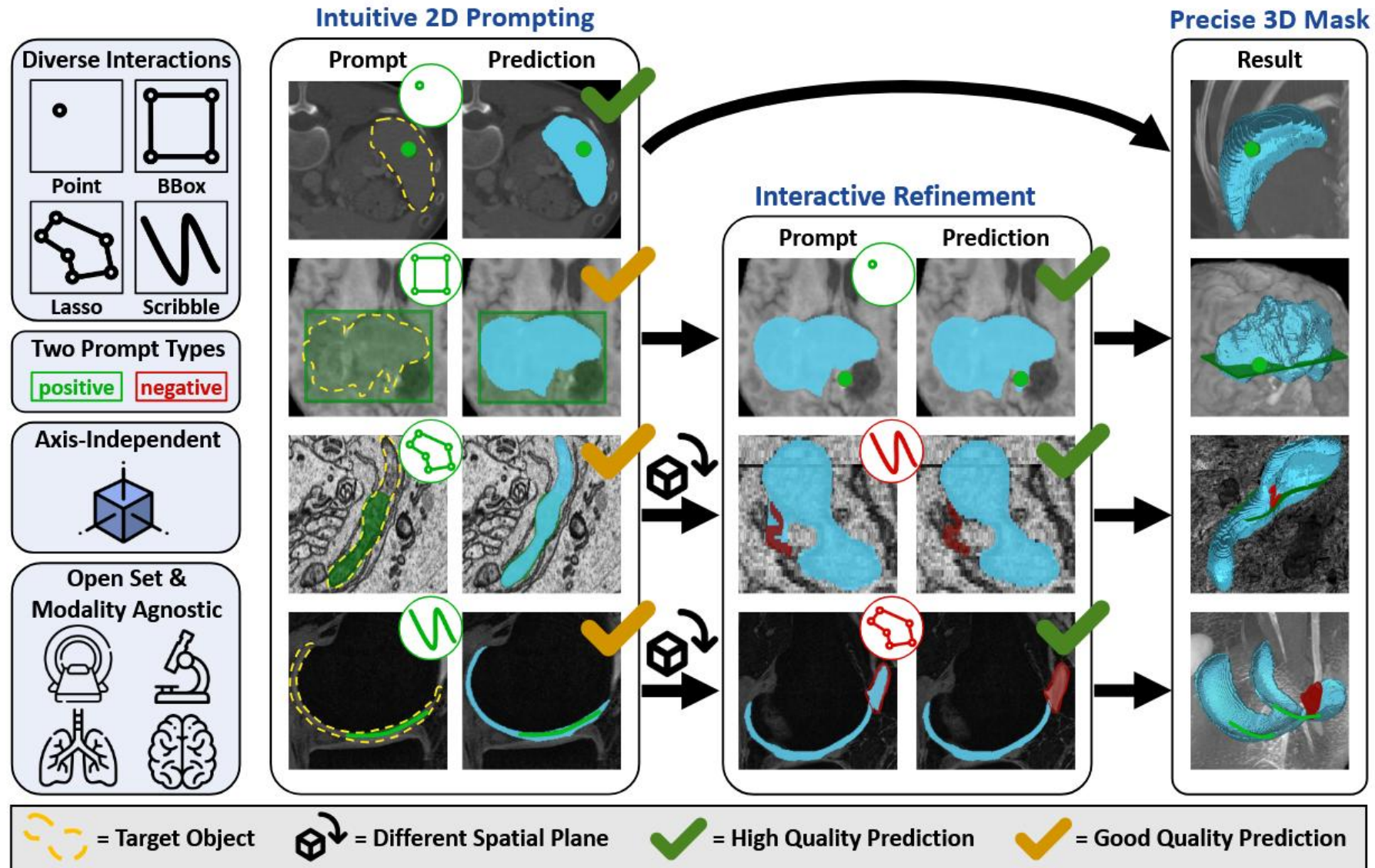
- `pip install napari-animation` // install Napari plugin to create videos

- Visualize image reconstruction in 3D, what about scaling?
- Create an animation with sections in 2D and sections in 3D, with and without annotated cells, with and without image, zooming in, zooming out, rotating, ...



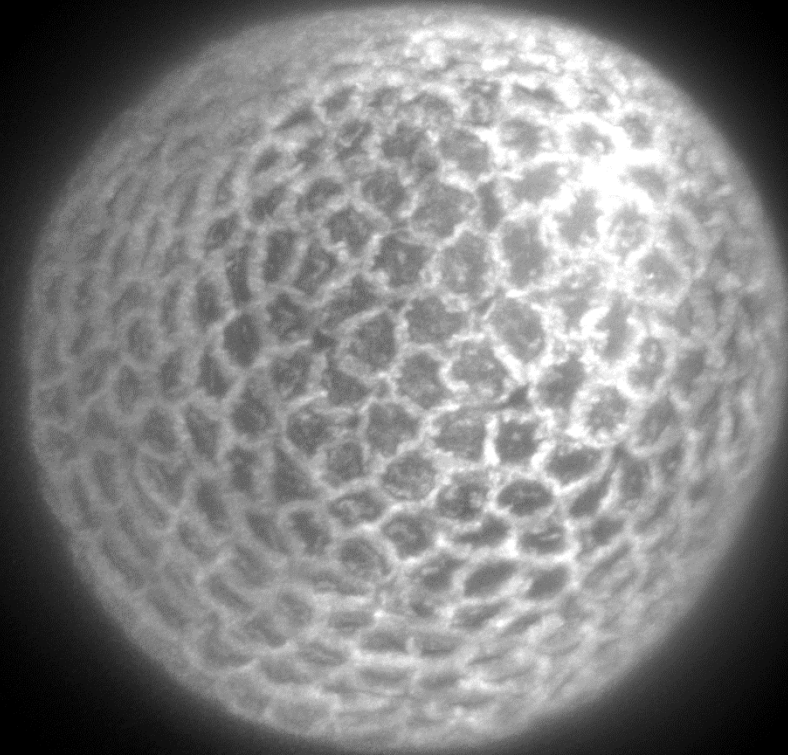
nnInteractive: Redefining 3D Promptable Segmentation

Fabian Isensee^{1,4*}, Maximilian Rokuss^{1,2*}, Lars Krämer^{1,4*}, Stefan Dinkelacker¹, Ashis Ravindran¹, Florian Stritzke⁵, Benjamin Hamm^{1,3}, Tassilo Wald^{1,2,4}, Moritz Langenberg^{1,2}, Constantin Ulrich¹, Jonathan Deissler^{1,2}, Ralf Floca¹, Klaus Maier-Hein^{1,2,3,4,6,7}

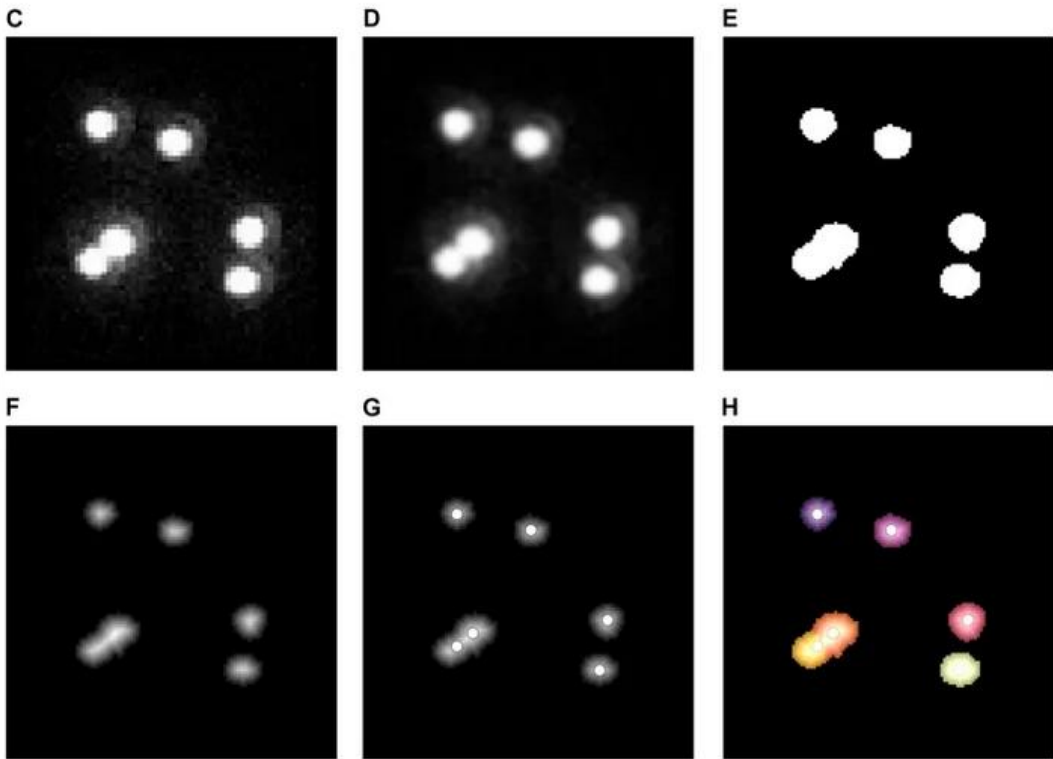
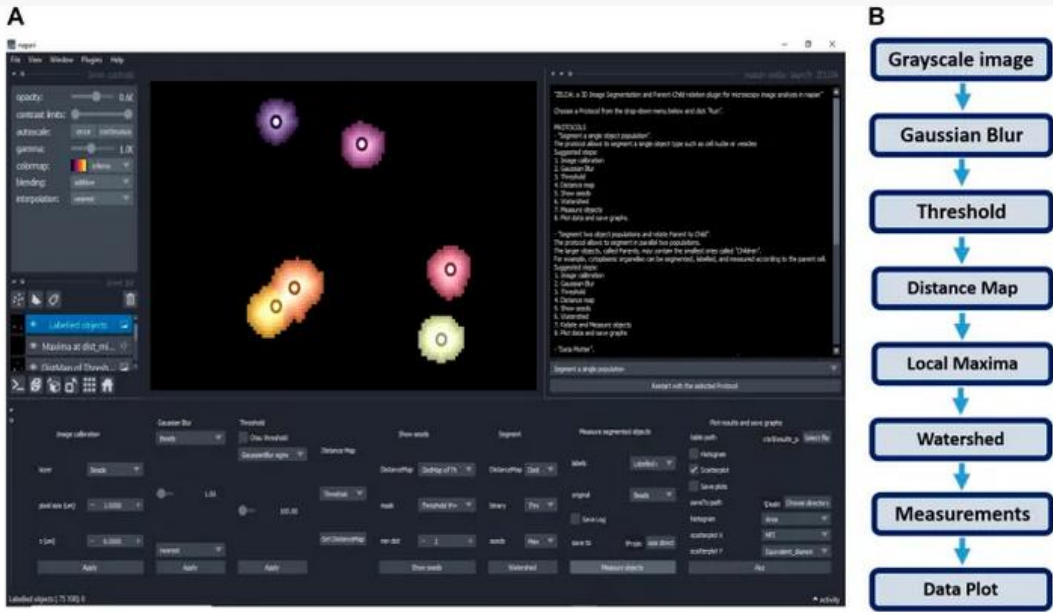


In a Miniforge **prompt**:

- `pip install napari-nninteractive` `// install nnInteractive`
- Use nnInteractive to interactively segment cells, play with different views for prompts
- Create an animation with sections in 2D and sections in 3D, with and without annotated cells, with and without image, zooming in, zooming out, rotating, ...

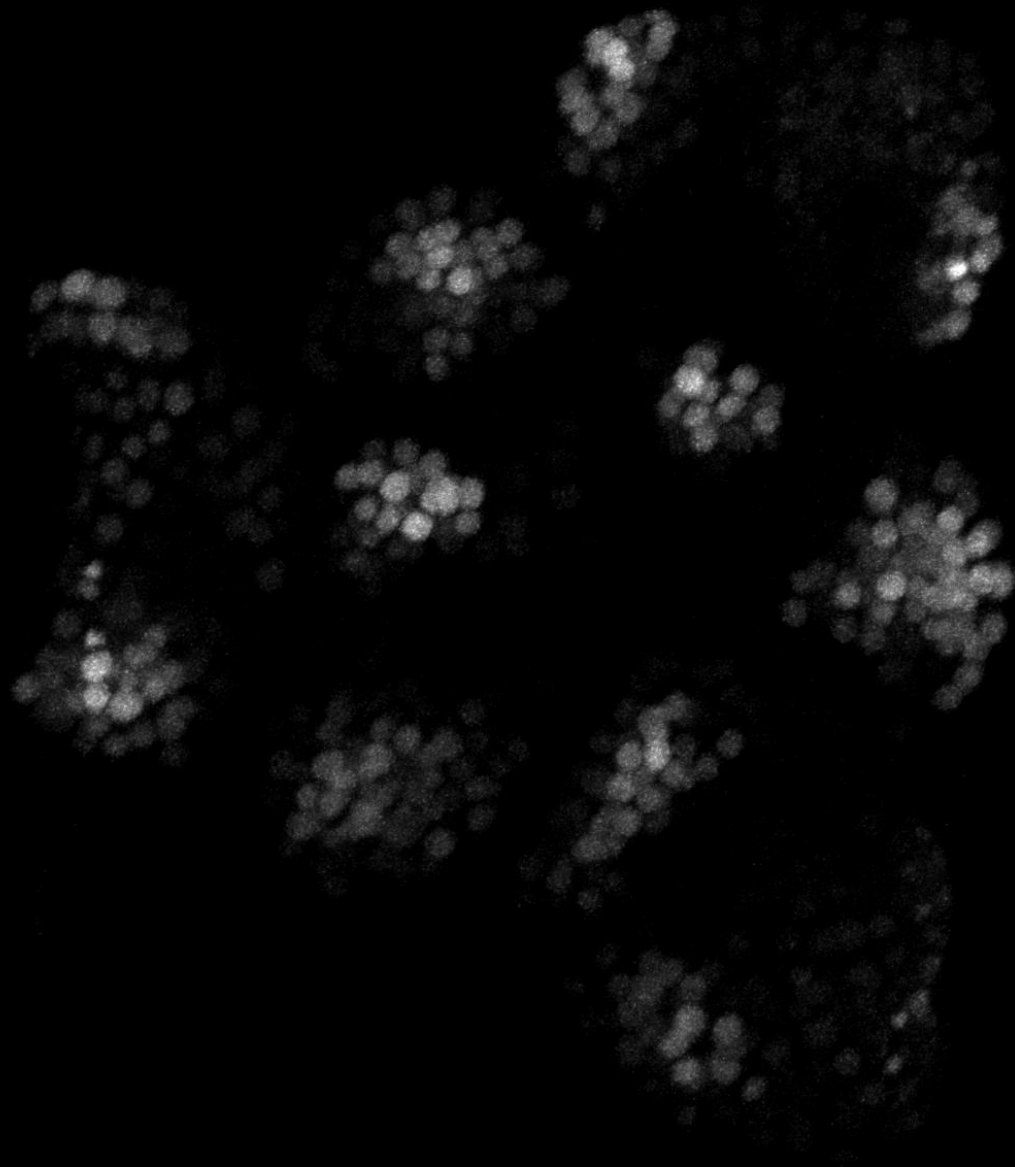


ZELDA: A 3D Image Segmentation and Parent-Child Relation Plugin for Microscopy Image Analysis in *napari*



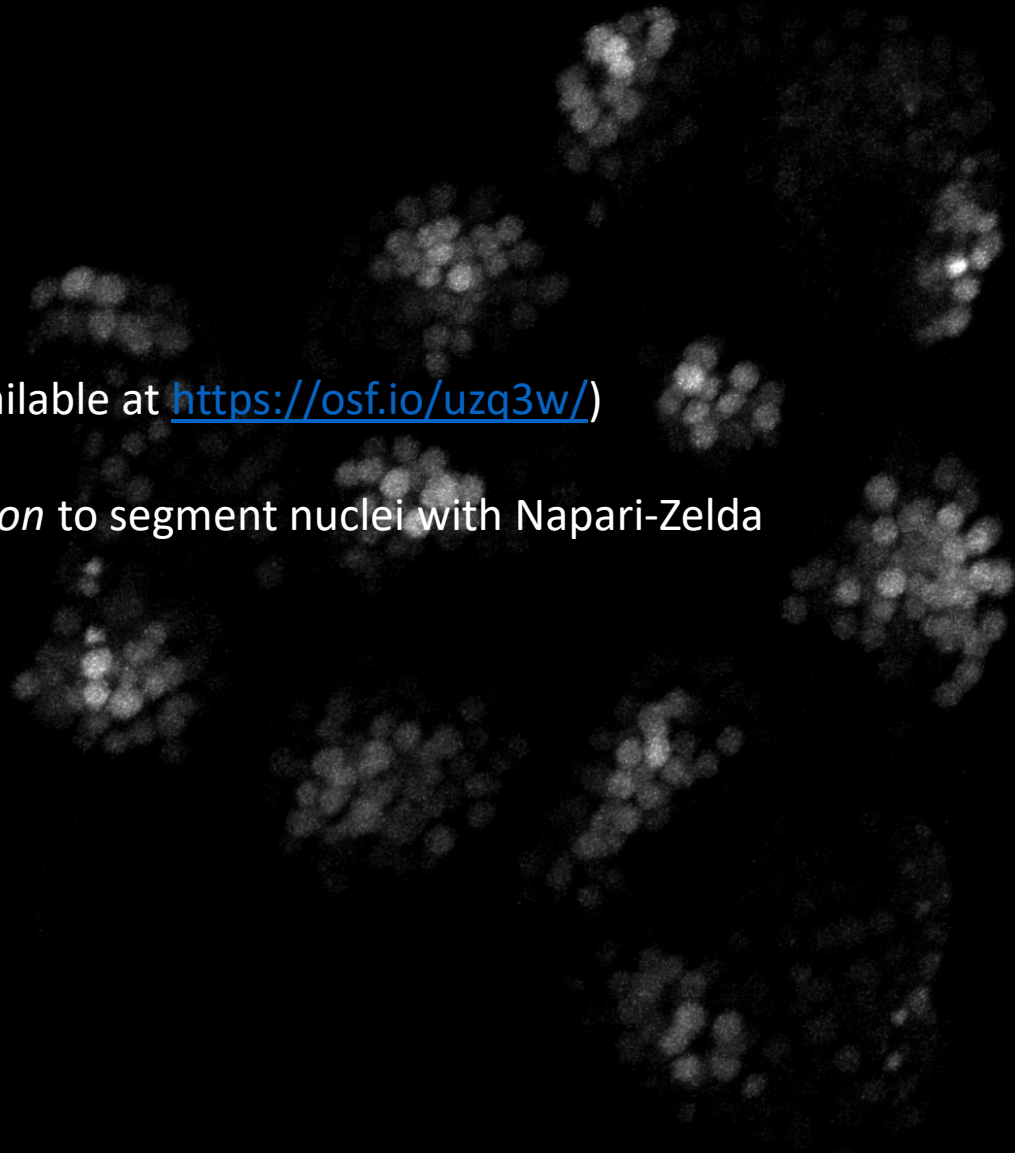
In a Miniforge **prompt**:

- `pip install napari-zelda`

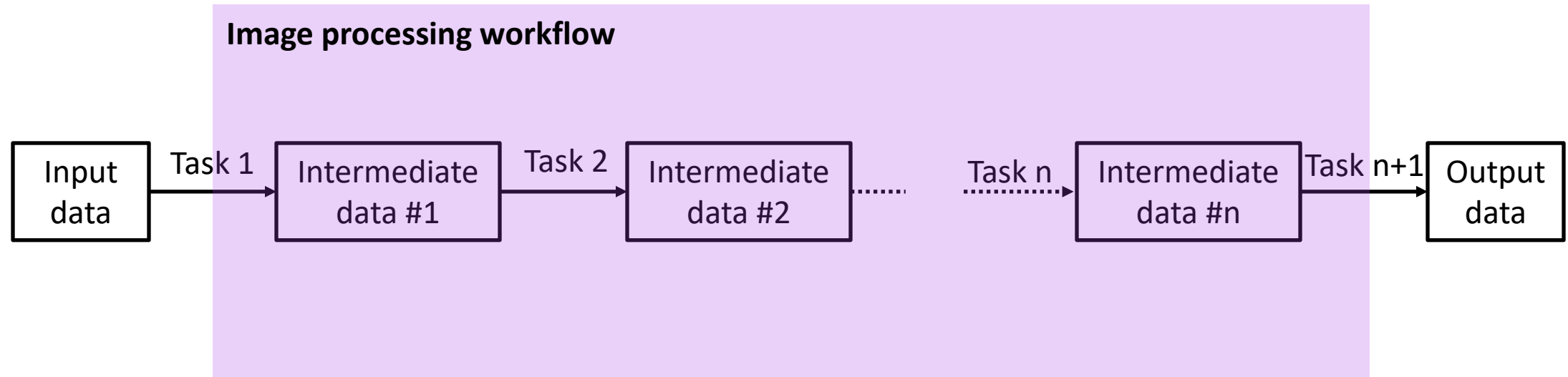


In a Miniforge **prompt**:

- pip install napari-zelda
- Open MutX_DR5v2_6_4_M1_nuclei.tif (available at <https://osf.io/uzq3w/>)
- Use the protocol *Segment a single population* to segment nuclei with Napari-Zelda



EXPLICIT PROGRAMMING



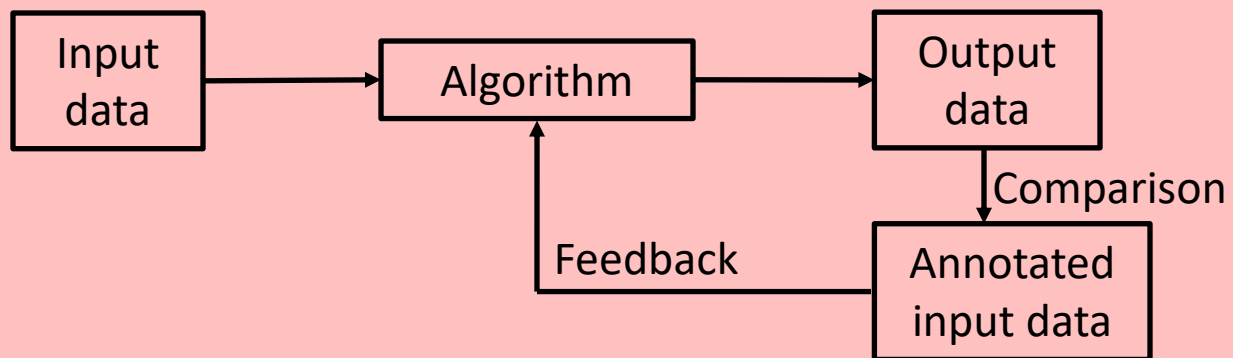
Input image

Nuclei



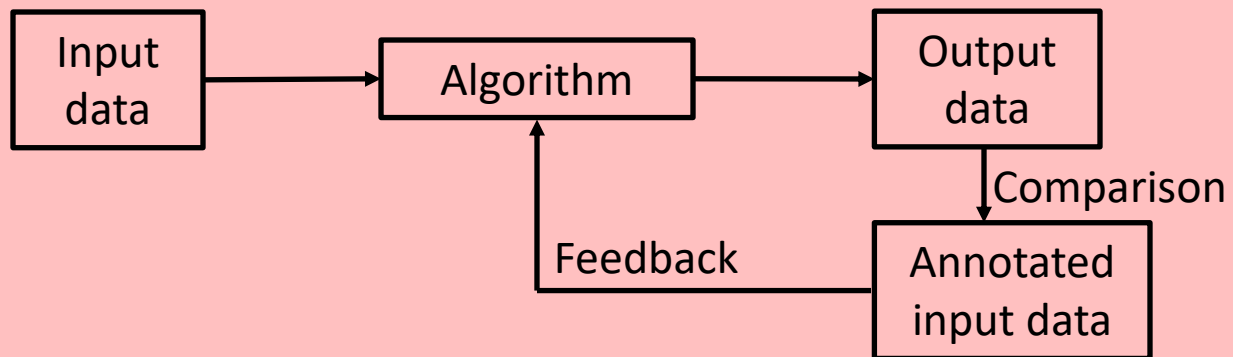
SUPERVISED MACHINE LEARNING

Supervised Learning

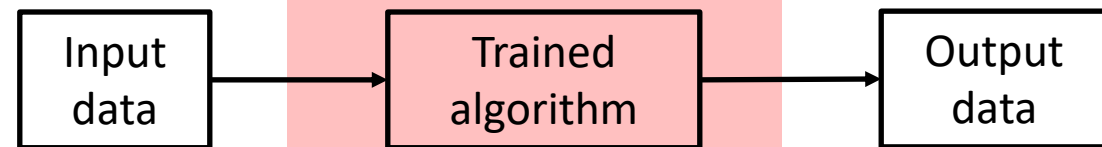


SUPERVISED MACHINE LEARNING

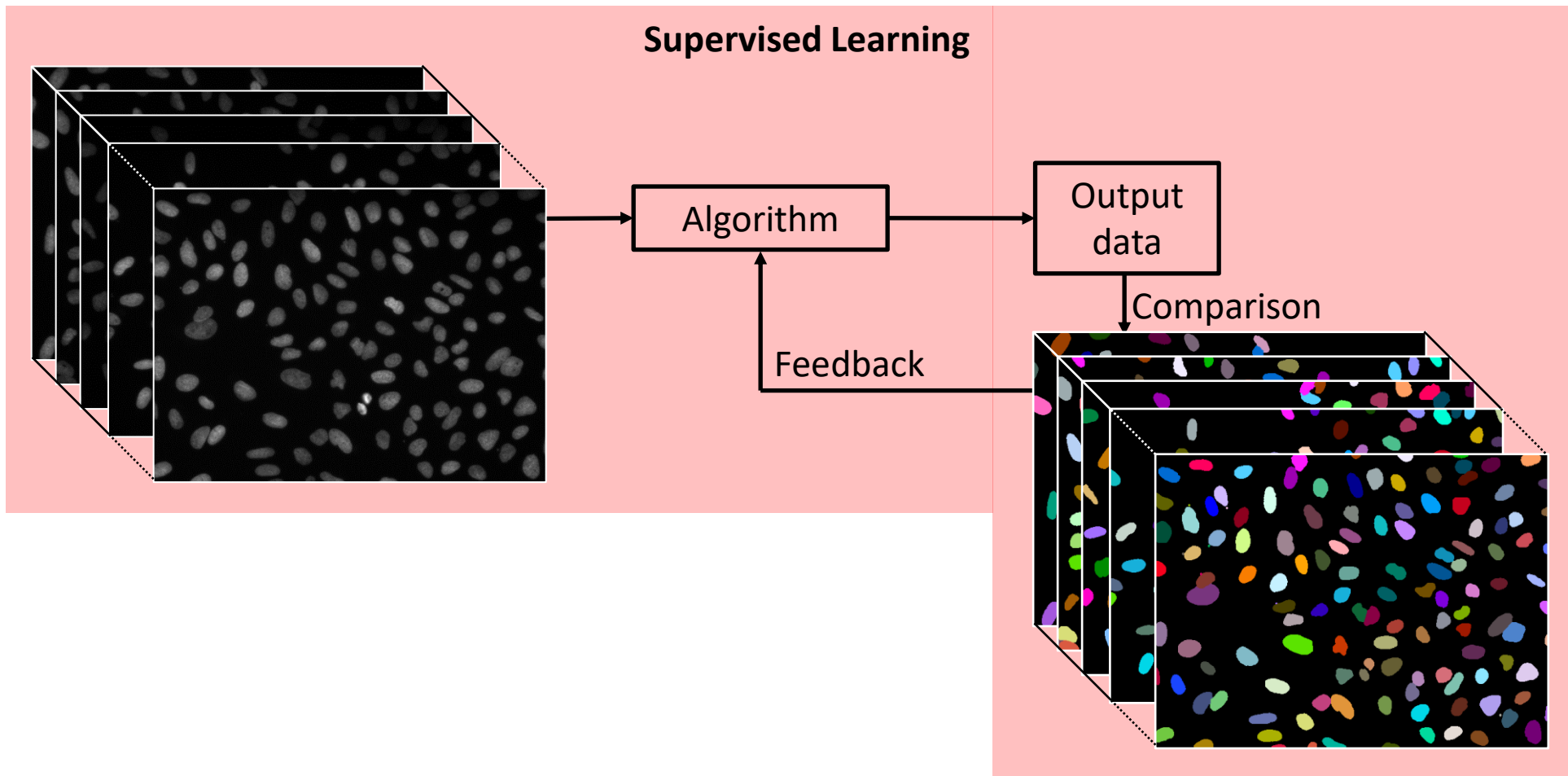
Supervised Learning



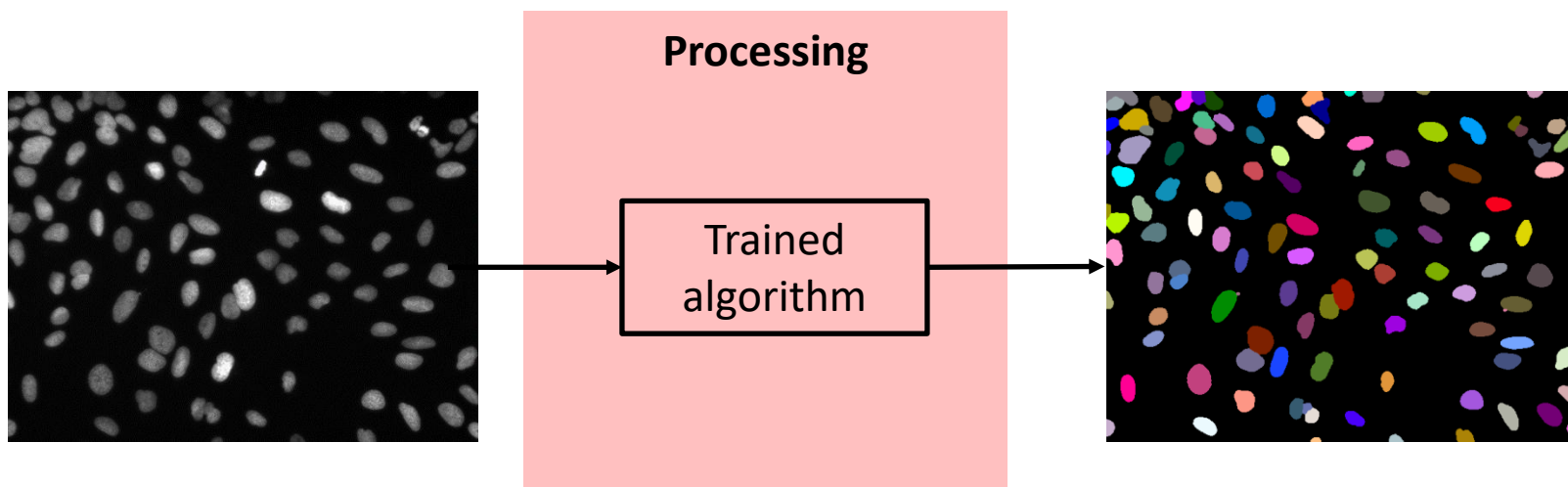
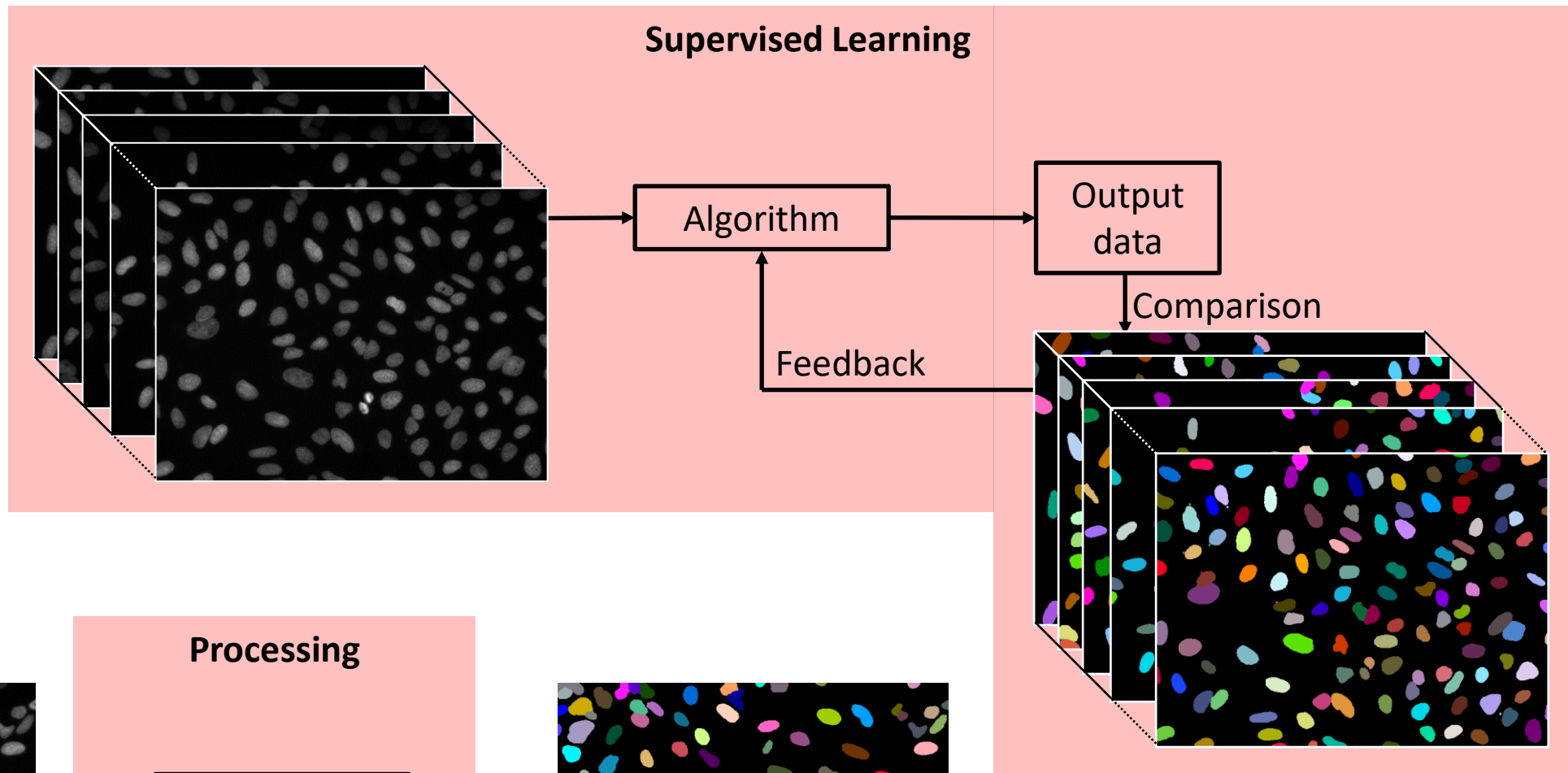
Processing



SUPERVISED MACHINE LEARNING



SUPERVISED MACHINE LEARNING

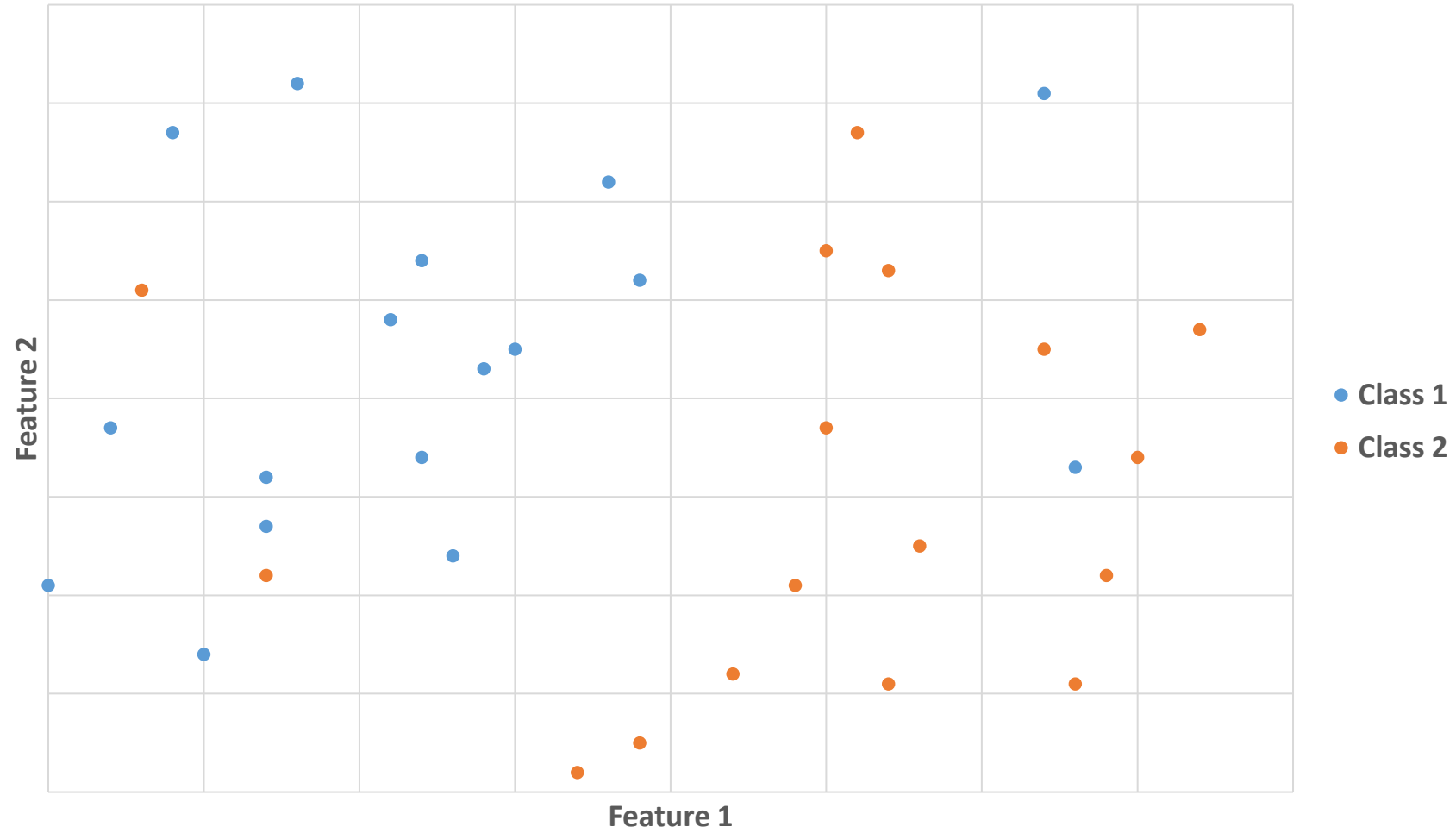


SHALLOW MACHINE LEARNING FOR PIXEL CLASSIFICATION

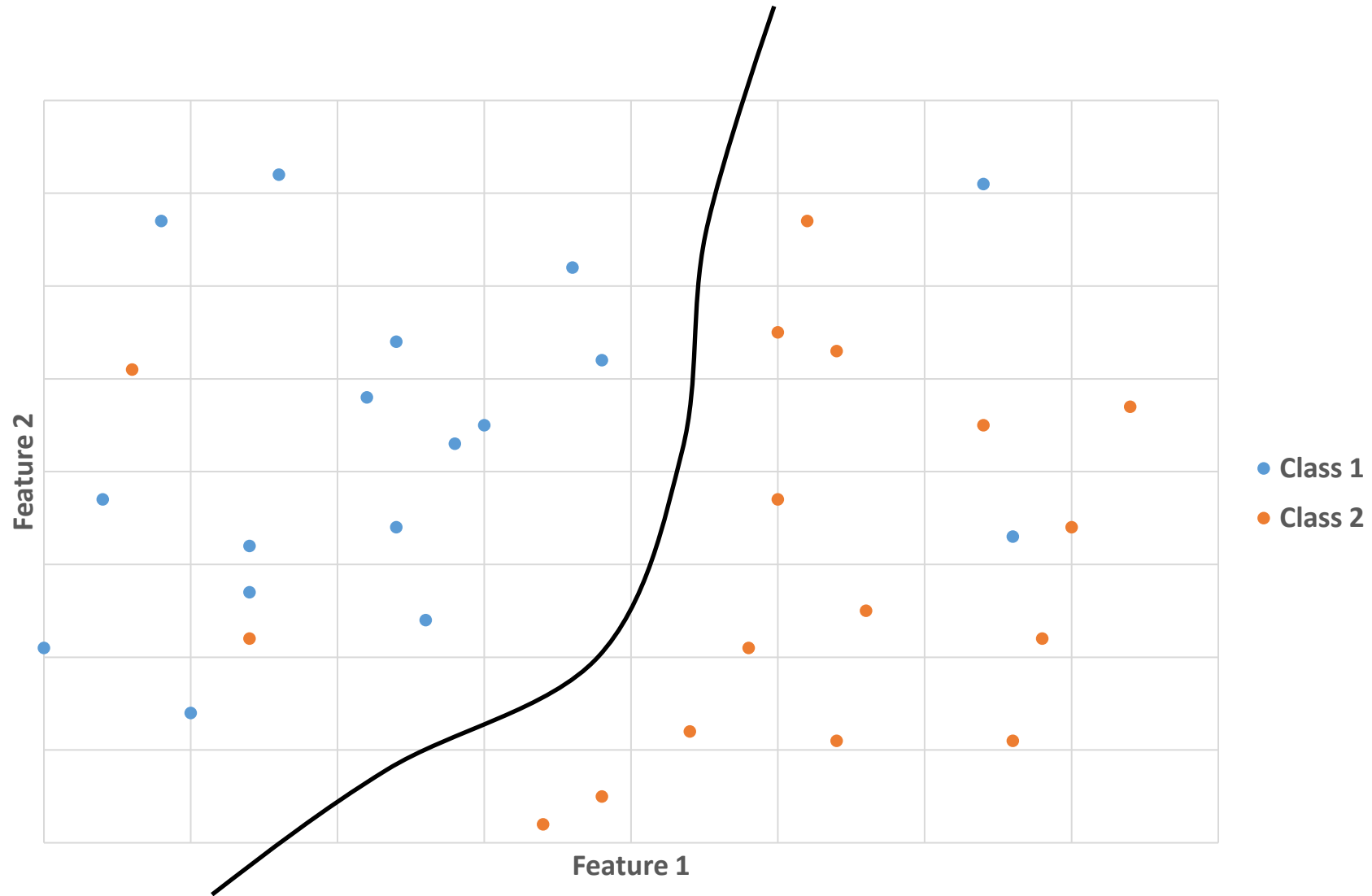
Supervised classification:

- **Examples** of classes are **manually** defined by the user
- A **classifier** is **trained** by using **defined features** with these examples
- Data is then **automatically classified** by using the trained classifier

SHALLOW MACHINE LEARNING FOR PIXEL CLASSIFICATION

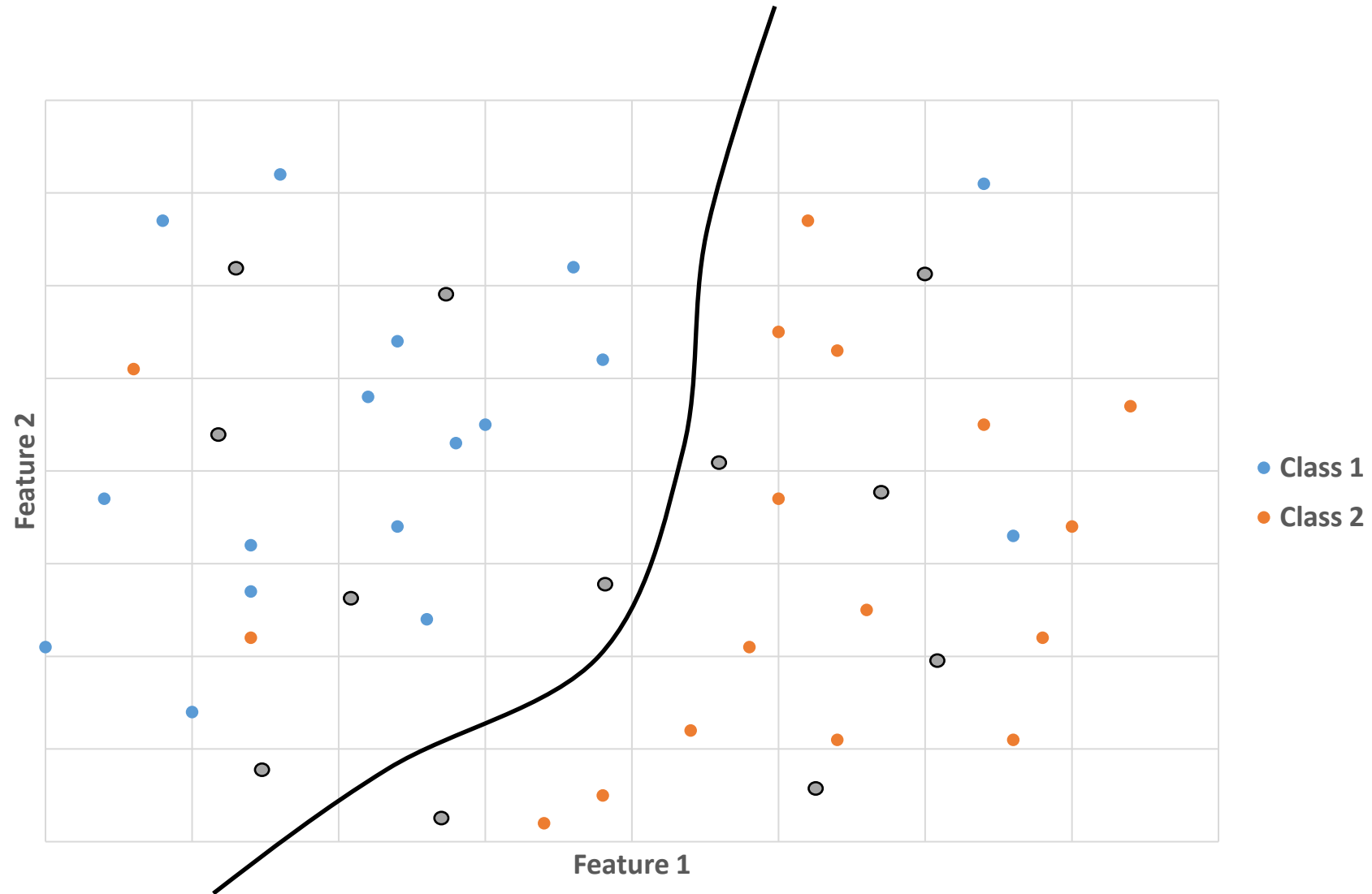


SHALLOW MACHINE LEARNING FOR PIXEL CLASSIFICATION



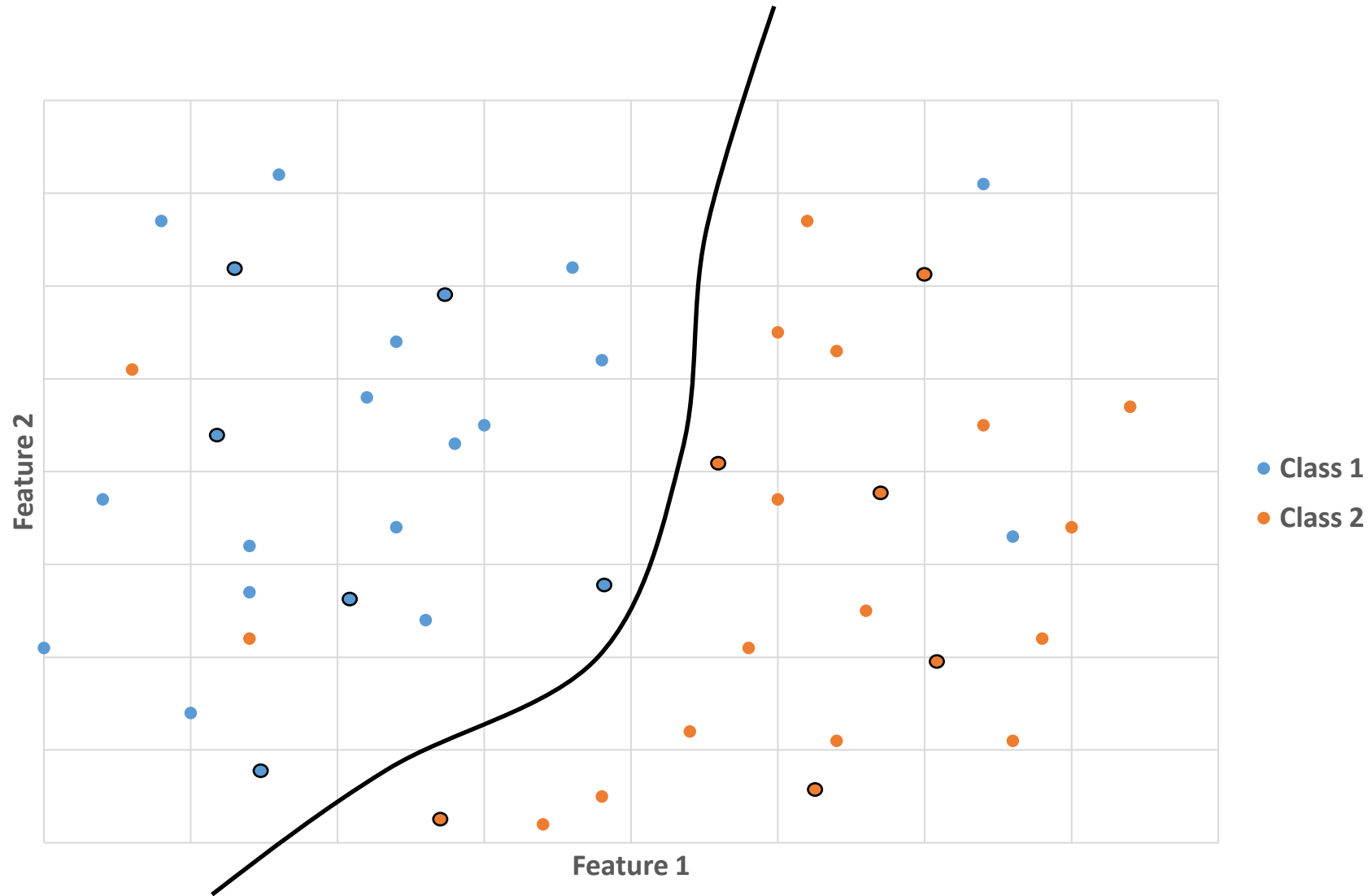
Training a classifier consists in **estimating** the "**best**" separation between classes

SHALLOW MACHINE LEARNING FOR PIXEL CLASSIFICATION



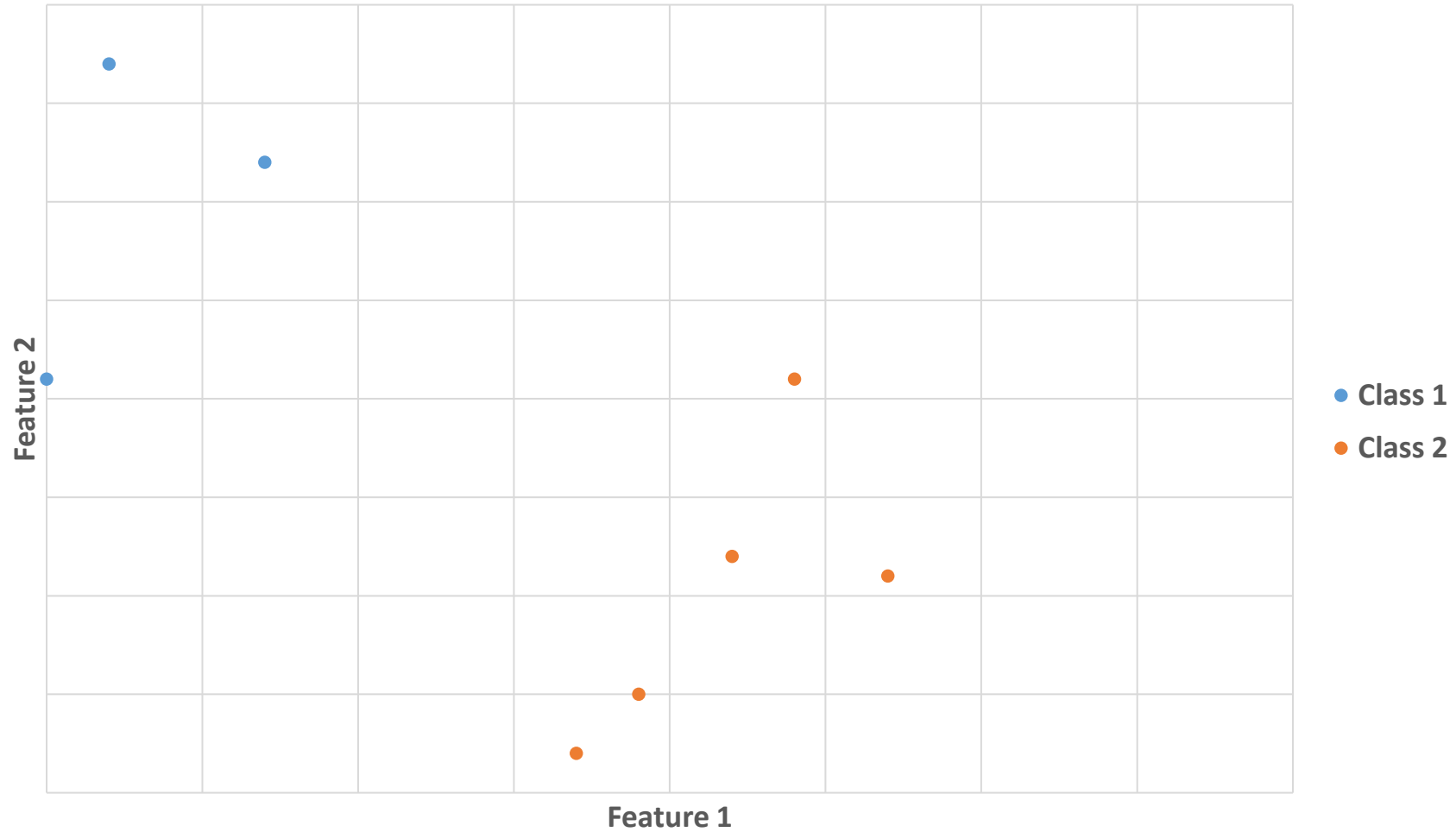
The **estimated class** for new data will be given by the **trained classifier**

SHALLOW MACHINE LEARNING FOR PIXEL CLASSIFICATION



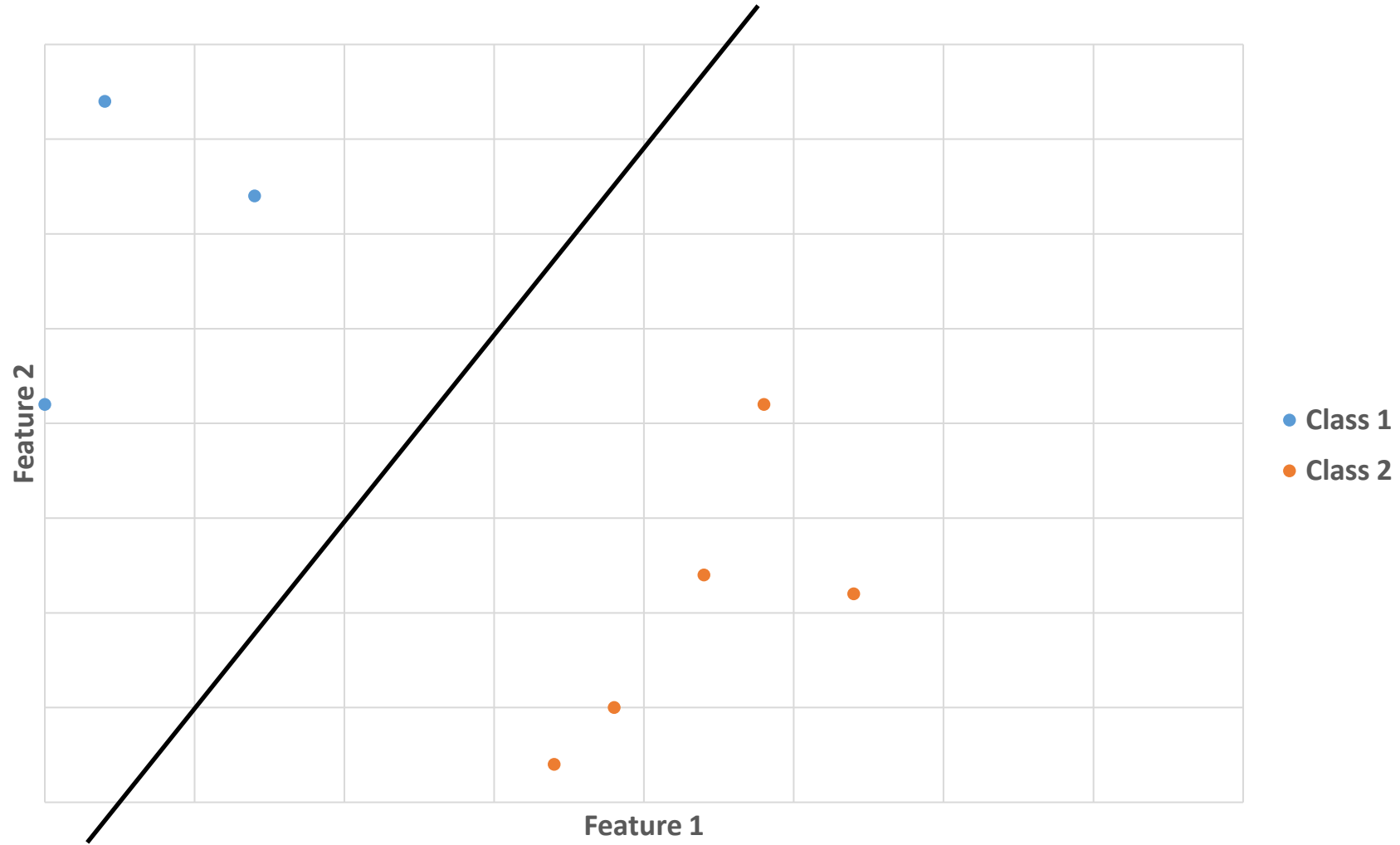
The **estimated class** for new data will be given by the **trained classifier**

SHALLOW MACHINE LEARNING FOR PIXEL CLASSIFICATION



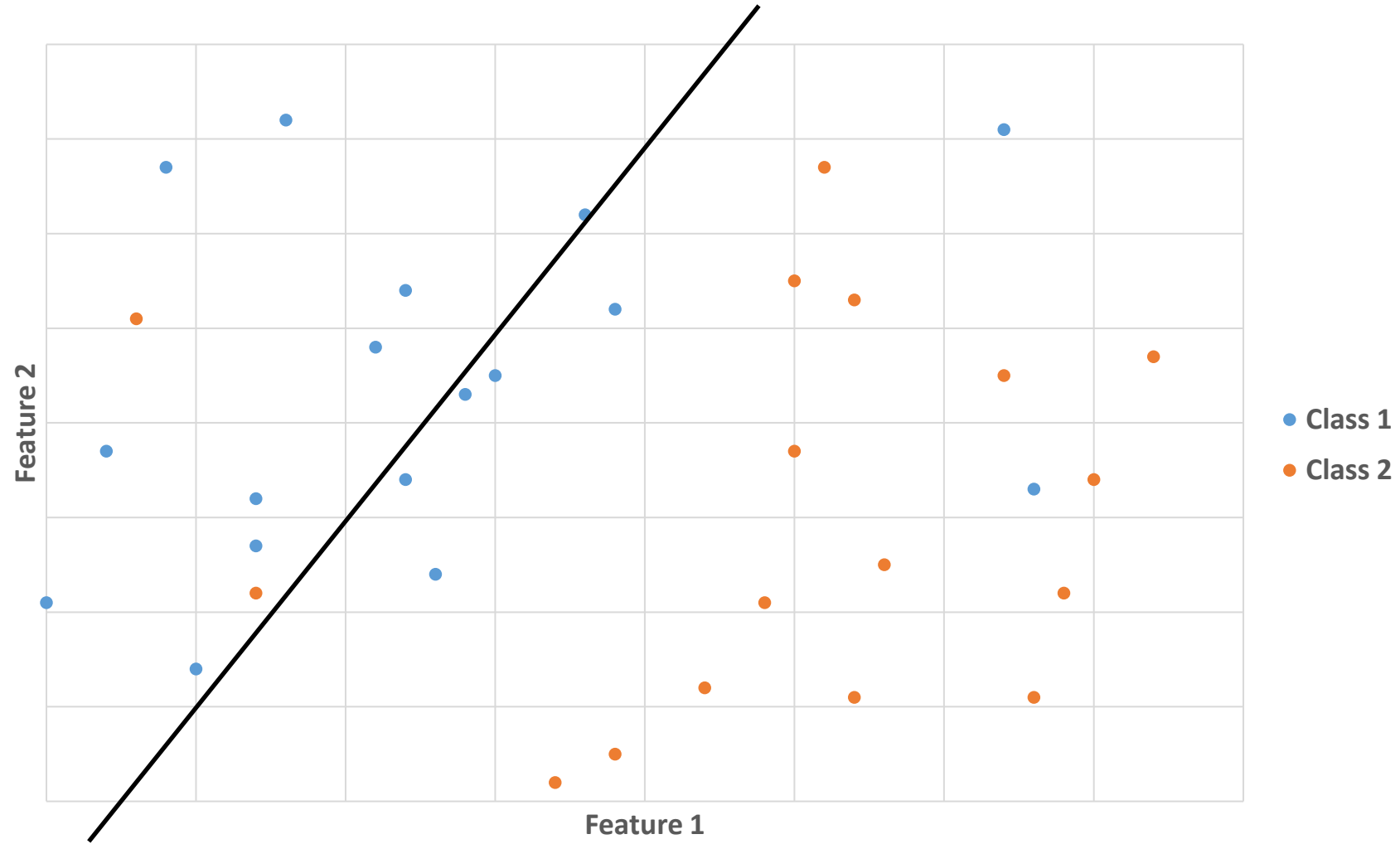
Training will be easier if **enough** and **representative** data is used

SHALLOW MACHINE LEARNING FOR PIXEL CLASSIFICATION



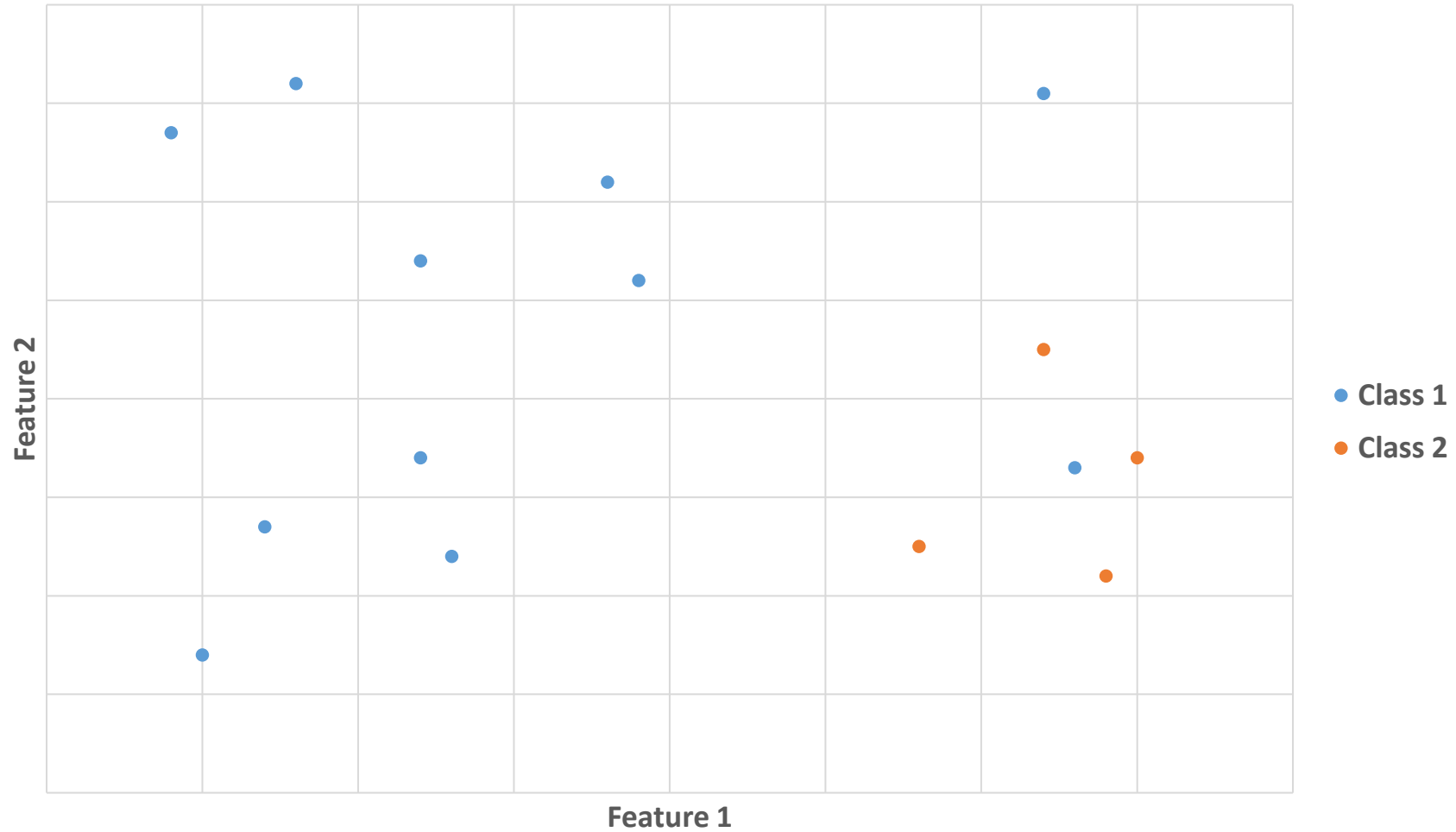
Training will be easier if **enough** and **representative** data is used

SHALLOW MACHINE LEARNING FOR PIXEL CLASSIFICATION



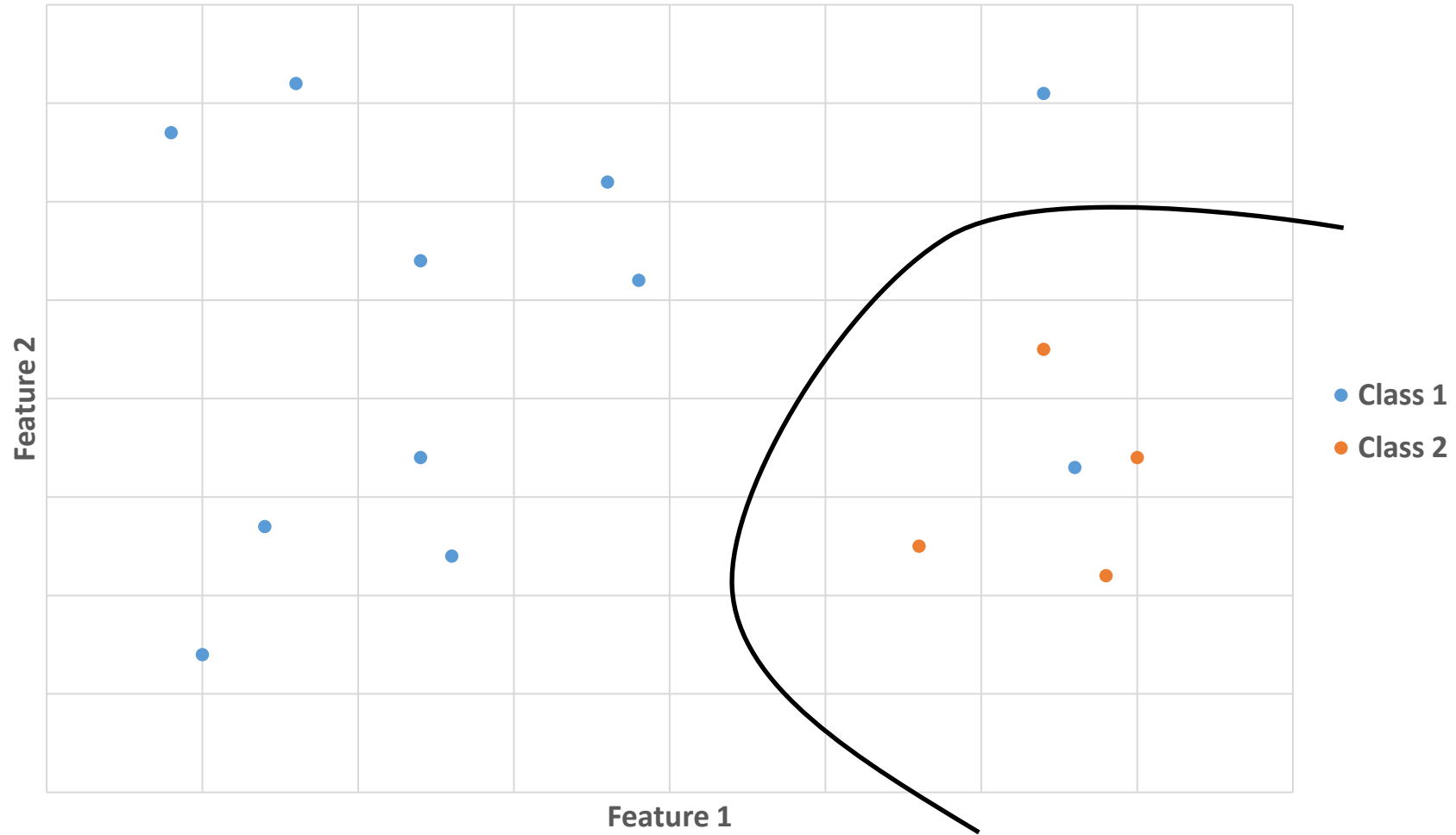
Training will be easier if **enough** and **representative** data is used

SHALLOW MACHINE LEARNING FOR PIXEL CLASSIFICATION



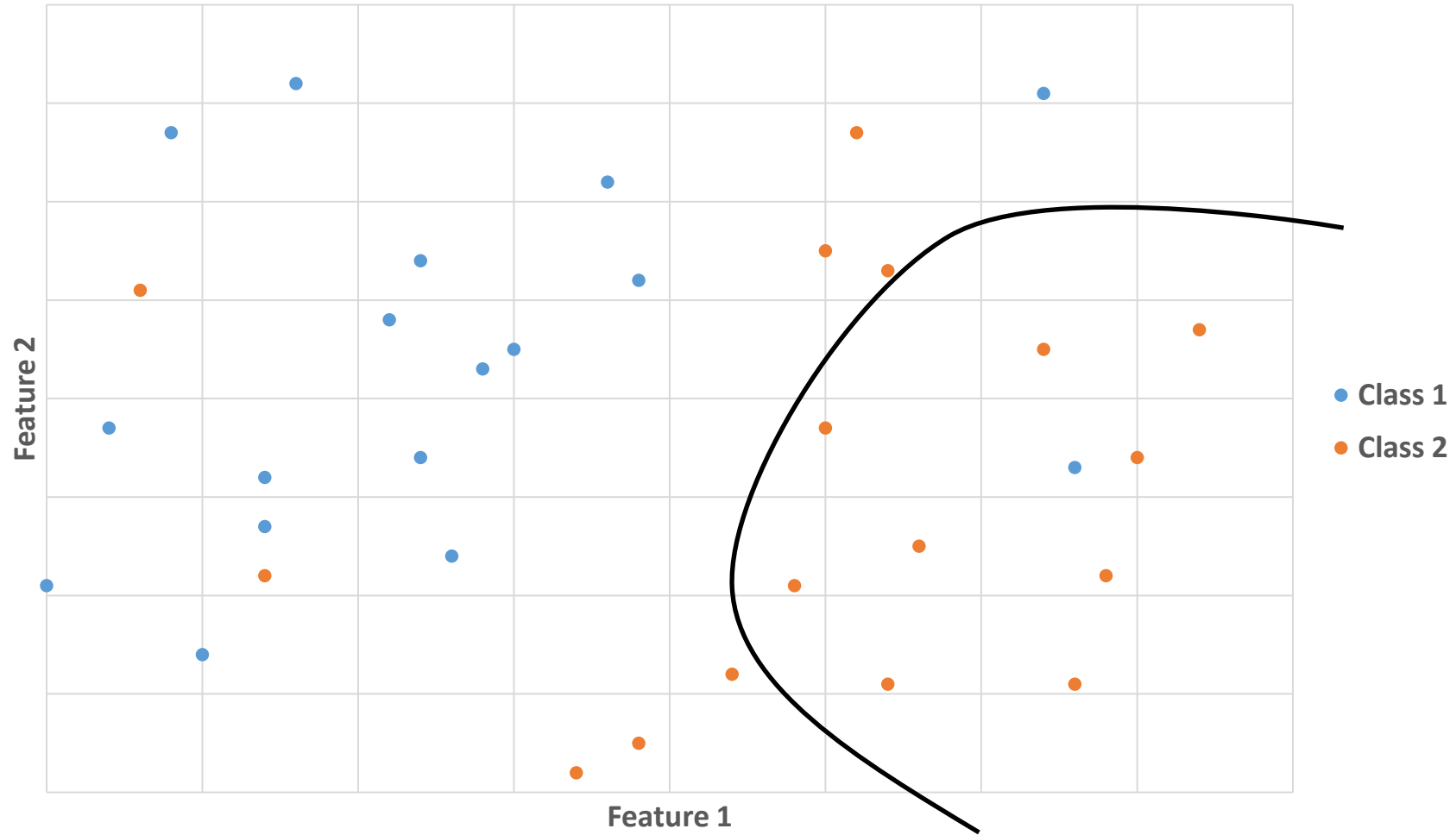
Class imbalance makes the training **more difficult**

SHALLOW MACHINE LEARNING FOR PIXEL CLASSIFICATION

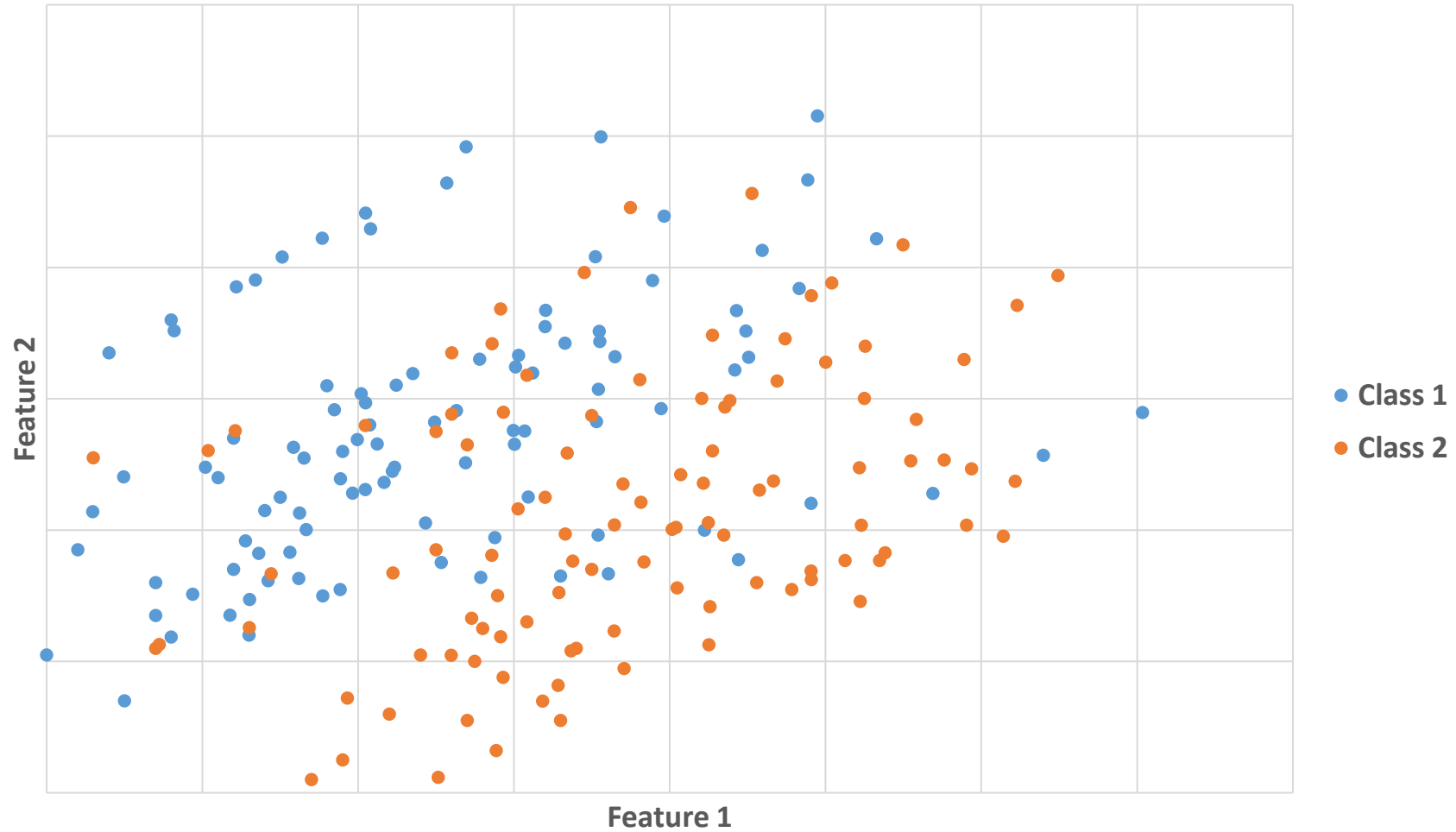


Class imbalance makes the training **more difficult**

SHALLOW MACHINE LEARNING FOR PIXEL CLASSIFICATION

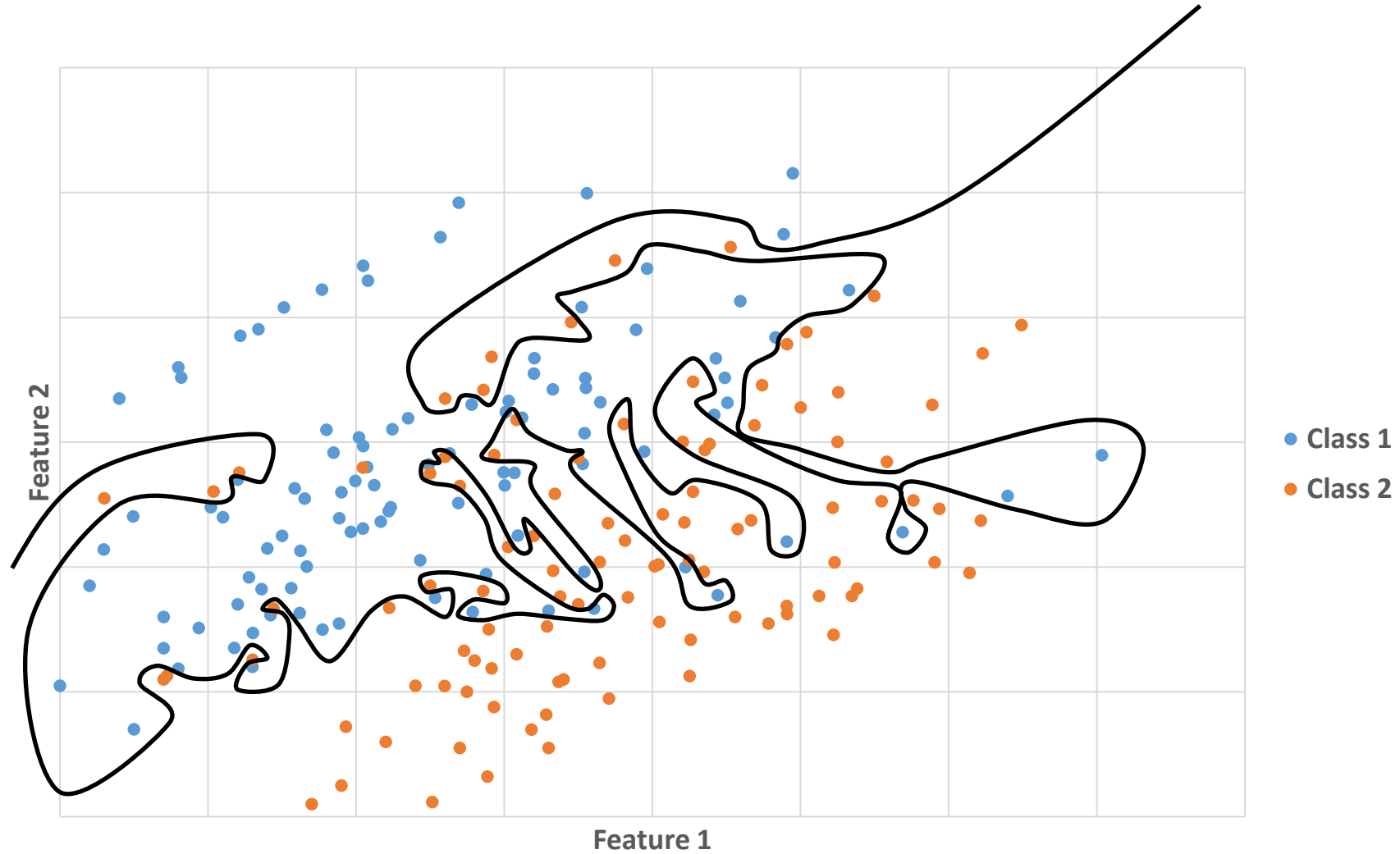


SHALLOW MACHINE LEARNING FOR PIXEL CLASSIFICATION



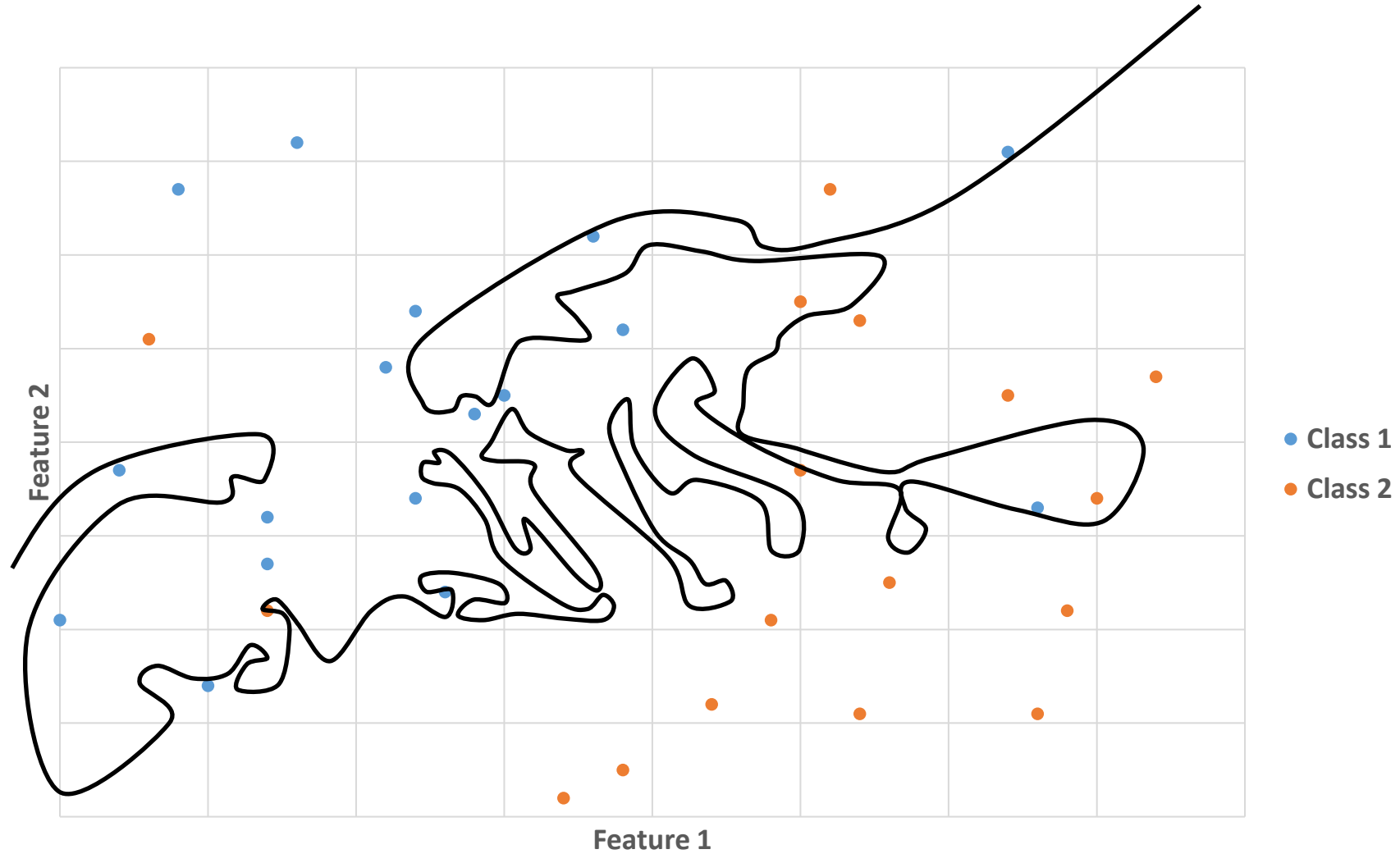
Too many annotations can lead to **over-fitting**: works great on training data but poorly on new data

SHALLOW MACHINE LEARNING FOR PIXEL CLASSIFICATION



Too many annotations can lead to **over-fitting**: works great on training data but poorly on new data

SHALLOW MACHINE LEARNING FOR PIXEL CLASSIFICATION



Too many annotations can lead to **over-fitting**: works great on training data but poorly on new data

SHALLOW MACHINE LEARNING FOR PIXEL CLASSIFICATION

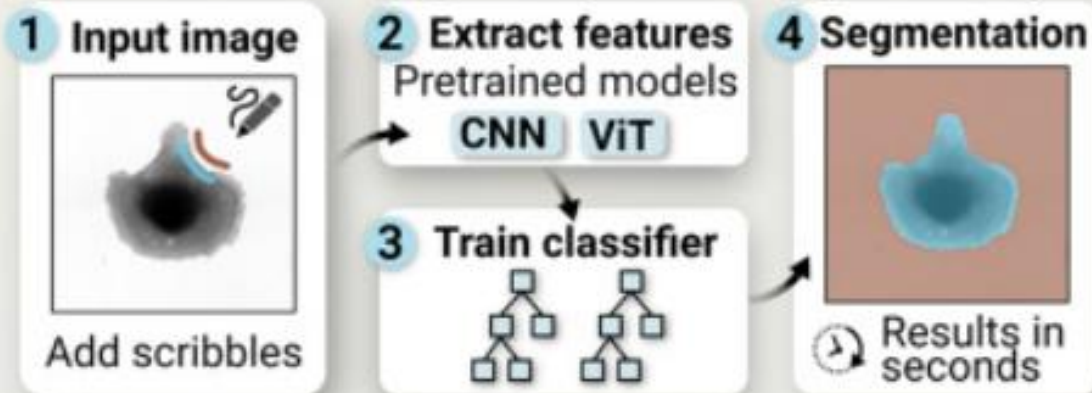
- Select **image features appropriate** to the classification problem
- Manually annotate regions/objects that are **representative** of what is seen in images
- Use **small lines** for annotations to **avoid over-representation**
- Define roughly the **same amount** of annotations for **each class**
- **Do not** manually annotate an **entire region of slide** to avoid over-fitting

Article

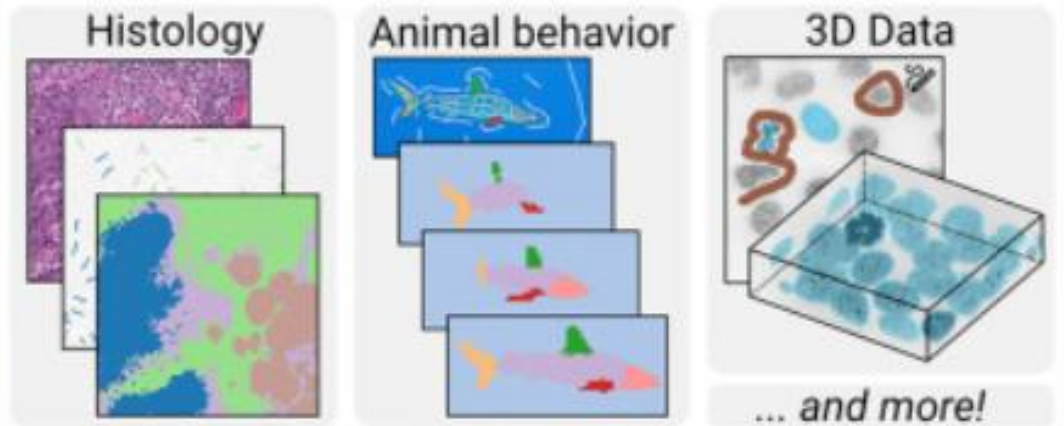
Convpaint—Interactive pixel classification using pretrained neural networks

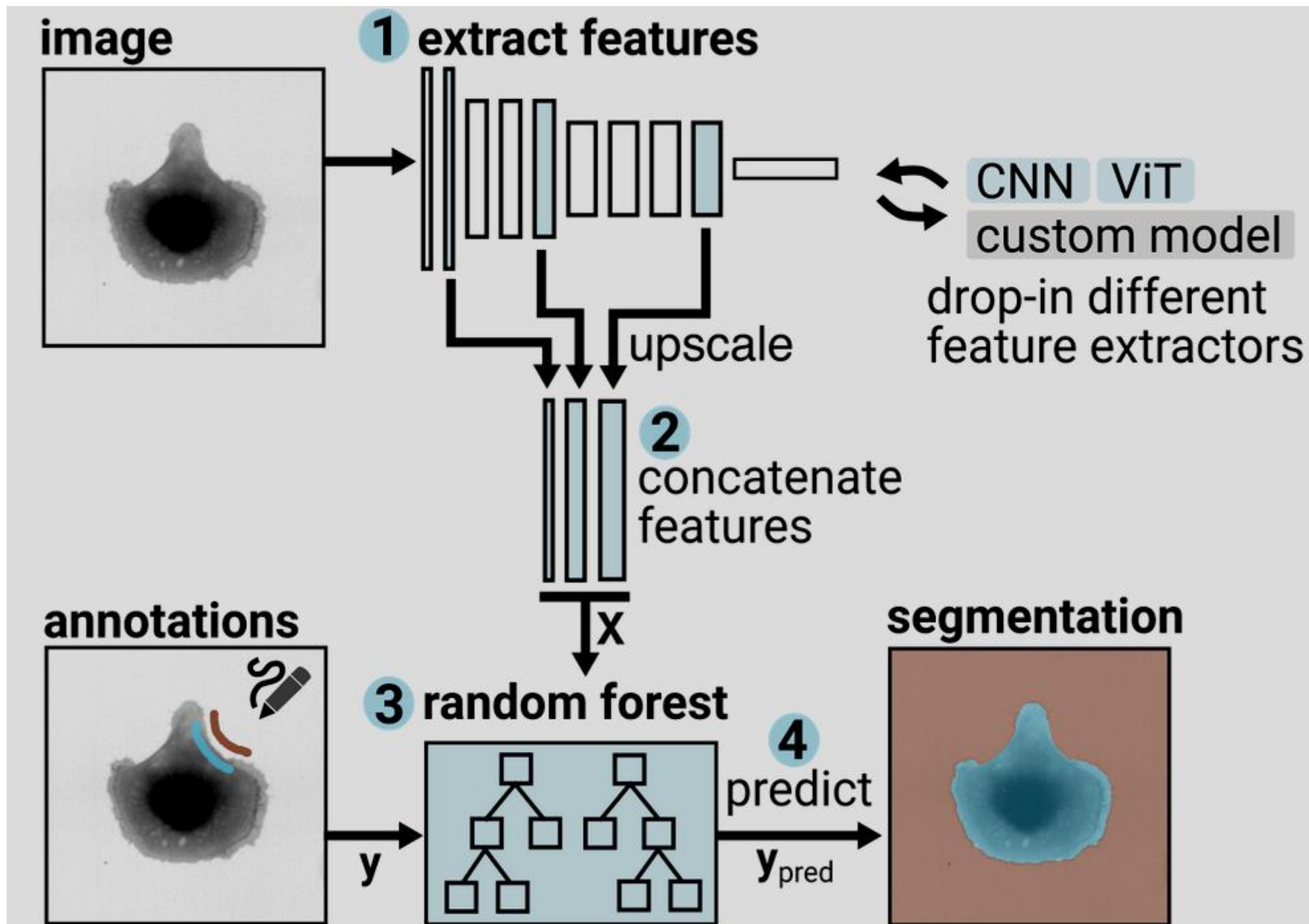
Lucien Hinderling^{1,2}  , Roman Schwob³, Guillaume Witz³  , Ana Stojiljković³,
Maciej Dobrzyński¹, Mykhailo Vladymyrov³, Joël Frei¹, Benjamin Grädel^{1,2}, Agne Frismantiene¹,
Olivier Pertz^{1,4}  

Convpaint



Applicable to many tasks:





- Open MutX_DR5v2_6_4_M1_cells.tif and MutX_DR5v2_6_4_M1_nuclei.tif (available at <https://osf.io/uzq3w/>), change colors, visualize both channels
- Use ConvPaint to segment nuclei
- Use ConvPaint to segment cells

